

УНИВЕРЗИТЕТ У БЕОГРАДУ
ЕЛЕКТРОТЕХНИЧКИ ФАКУЛТЕТ



**ИНТЕРАКТИВНА ВИЗУЕЛИЗАЦИЈА ГРАФОВСКИХ
АЛГОРИТАМА ЗА ПРОНАЛАЖЕЊЕ ПУТА СА
ПОДРШКОМ ВЕШТАЧКЕ ИНТЕЛИГЕНЦИЈЕ**

Мастер рад

Ментор:
проф. др Марија Пунт

Кандидат:
Предраг Пешић 2024/3281

Београд, август 2026.

САЖЕТАК

Алгоритми за проналажење пута предају се на већини рачунарских студијских програма, али студенти теже стичу интуицију о томе зашто се два поступка различито понашају на истом улазу. Постојећи алати за визуелизацију приказују рад алгоритама, али ништа не мере и не омогућавају упоредно покретање више поступака над истом мапом. Корисника остављају у улози посматрача, што према објављеним истраживањима даје слабе исходе учења.

Овај рад представља интерактивну веб апликацију која те три празнине попуњава. Апликација визуелизује осам алгоритама на мрежном графу, приказује мерене величине током рада, омогућава упоредно покретање и садржи режим у коме корисник сам конструише пут и добија оцену. Уз то је развијен слој заснован на великим језичким моделима. Његова улога ограничена је на разумевање захтева и на објашњавање већ проверених резултата, док сваку бројчану тврдњу израчунава сам систем.

Систем је вреднован на скоро 9500 симулација алгоритама, више од 11000 симулација бодовног система и око 400 позива упућених ка пет језичких модела. Уведене су и две мере које се у уобичајеним приказима ове материје не срећу. Прва је заједничка скала цене пута, која омогућава да се упореде поступци што растојање рачунају на различите начине. Друга је субоптималност, изражена као проценат за који је пронађени пут скупљи од најјефтинијег.

Мерења су дала три истакнута налаза. Претрага у ширину даје путеве скупље од најјефтинијих за 18,1 одсто на мапама са пондерисаним тереном. Претрага у дубину троши пет пута више меморије од претраге у ширину, супротно закључку изведеном за стабло претраге. Алгоритам A^* даје исти резултат као Дајкстрин алгоритам уз 62,5 одсто мање обраде. Испитивање језичких модела показује да њихова тачност у читавању екстремне вредности из табеле износи између 35,7 и 90,0 одсто, чиме је потврђена оправданост усвојене поделе одговорности.

Кључне речи: проналажење пута, визуелизација алгоритама, алгоритам A^* , хеуристичка претрага, велики језички модели, едукативни софтвер, вредновање перформанси, кориснички интерфејс.

САДРЖАЈ

САЖЕТАК	II
САДРЖАЈ	III
1. УВОД	1
2. АЛГОРИТМИ ЗА ПРОНАЛАЖЕЊЕ ПУТА НА МРЕЖНОМ ГРАФУ	2
2.1. ФОРМУЛАЦИЈА ПРОБЛЕМА	2
2.2. НЕИНФОРМИСАНА ПРЕТРАГА	2
2.3. ПРЕТРАГА СА ЦЕНОМ ПРЕЛАЗА	3
2.4. ИНФОРМИСАНА ПРЕТРАГА	3
2.5. ПОРОДИЦА ПОНДЕРИСАНИХ ВАРИЈАНТИ	4
2.6. ХЕУРИСТИЧКЕ ФУНКЦИЈЕ	4
2.7. УПОРЕДНИ ПРЕГЛЕД СВОЈСТАВА	5
3. ПРЕГЛЕД ПОСТОЈЕЋИХ РЕШЕЊА И КОРИШЋЕНЕ ТЕХНОЛОГИЈЕ	6
3.1. ПОСТОЈЕЋА РЕШЕЊА	6
3.2. УПОРЕДНА АНАЛИЗА И ИЗВЕДЕНИ ЗАХТЕВИ	7
3.3. КОРИШЋЕНЕ ТЕХНОЛОГИЈЕ	8
4. АРХИТЕКТУРА И ИМПЛЕМЕНТАЦИЈА СИСТЕМА	10
4.1. ОПШТА АРХИТЕКТУРА	10
4.2. ЈЕДИНСТВЕНИ ИНТЕРФЕЈС АЛГОРИТМА	11
4.3. ИМПЛЕМЕНТАЦИЈА АЛГОРИТАМА	11
4.4. ПРИКАЗ МРЕЖЕ И УПРАВЉАЊЕ РЕПРОДУКЦИЈОМ	12
4.5. ГЕНЕРАТОРИ МАПА	12
4.6. СЕРВЕРСКИ СЛОЈ, МОДЕЛ ПОДАКА И БЕЗБЕДНОСТ	13
5. РЕЖИМИ РАДА И КОРИСНИЧКО УПУТСТВО	14
5.1. ЗАЈЕДНИЧКИ РАСПОРЕД ЕКРАНА	14
5.2. РЕЖИМ ВИЗУЕЛИЗАЦИЈЕ	14
5.3. РЕЖИМ ПОРЕЂЕЊА	16
5.4. РЕЖИМ ПОЛИГОНА	17
5.5. ИНТЕРАКТИВНИ ВОДИЧ И КОРИСНИЧКИ НАЛОГ	19
6. ИНТЕГРАЦИЈА ВЕШТАЧКЕ ИНТЕЛИГЕНЦИЈЕ	21
6.1. ПРИСТУП И АРХИТЕКТУРА СЛОЈА	21
6.2. ТУТОР	22
6.3. ГЕНЕРАТОР СЦЕНАРИЈА	23
6.4. ПРЕПОРУЧИВАЧ И КОНТЕКСТУАЛНА ПОМОЋ	23
6.5. РОБУСНОСТ И ПОДЕЛА ОДГОВОРНОСТИ	24
7. МЕТОДОЛОГИЈА ЕВАЛУАЦИЈЕ	26
7.1. МЕРЕНЕ ВЕЛИЧИНЕ	26
7.2. ИНФРАСТРУКТУРА И КАТЕГОРИЈЕ МЕРЕЊА	27
7.3. УСЛОВИ МЕРЕЊА И ОБРАДА РЕЗУЛТАТА	27
7.4. ОГРАНИЧЕЊА МЕТОДОЛОГИЈЕ	28
8. РЕЗУЛТАТИ ЕВАЛУАЦИЈЕ АЛГОРИТАМА	29
8.1. ЗБИРНИ ПРЕГЛЕД	29
8.2. ПОНАШАЊЕ ПО ТИПОВИМА МАПА	30

8.3.	СКАЛАБИЛНОСТ.....	31
8.4.	УТИЦАЈ ХЕУРИСТИЧКЕ ФУНКЦИЈЕ И ТИПА СУСЕДСТВА	32
8.5.	УТИЦАЈ ТЕЖИНСКОГ ПАРАМЕТРА	34
8.6.	МЕМОРИЈСКИ ОТИСАК И ПРОВЕРА ОСНОВНЕ МЕРЕ	34
8.7.	РАСПРАВА.....	35
9.	ЕВАЛУАЦИЈА ВЕШТАЧКЕ ИНТЕЛИГЕНЦИЈЕ И КОРИСНИЧКОГ ИНТЕРФЕЈСА	37
9.1.	ВРЕДНОВАЊЕ КОМПОНЕНТЕ ВЕШТАЧКЕ ИНТЕЛИГЕНЦИЈЕ.....	37
9.1.1.	<i>Тачност препоруке и проблем изједначених резултата</i>	<i>37</i>
9.1.2.	<i>Генератор сценарија и тутор.....</i>	<i>39</i>
9.1.3.	<i>Време одзива и робусност.....</i>	<i>39</i>
9.2.	ВЕРИФИКАЦИЈА БОДОВНОГ СИСТЕМА.....	40
9.3.	ВРЕДНОВАЊЕ КОРИСНИЧКОГ ИНТЕРФЕЈСА	41
9.4.	ПРИПРЕМЉЕНИ ИНСТРУМЕНТ ЗА КОРИСНИЧКО ТЕСТИРАЊЕ.....	42
9.5.	ОГРАНИЧЕЊА СПРОВЕДЕНОГ ВРЕДНОВАЊА	42
10.	ЗАКЉУЧАК	44
10.1.	ПРАВЦИ ДАЉЕГ РАЗВОЈА	45
	ЛИТЕРАТУРА.....	46
	СПИСАК СКРАЋЕНИЦА	49
	СПИСАК СЛИКА.....	50
	СПИСАК ТАБЕЛА.....	51
	ПРИЛОГ А: СТРУКТУРА ПРОЈЕКТА И УПУТСТВО ЗА ПОКРЕТАЊЕ	52
A.1.	ЗАХТЕВИ, ПОДЕШАВАЊЕ И ПОКРЕТАЊЕ	52
A.2.	ФОРМАТИ УЛАЗНИХ И ИЗЛАЗНИХ ДАТОТЕКА.....	52
	ПРИЛОГ Б: НАЈВАЖНИЈИ ДЕЛОВИ ПРОГРАМСКОГ КОДА	53
B.1.	ЈЕДИНСТВЕНИ ИНТЕРФЕЈС АЛГОРИТМА	53
B.2.	ХЕУРИСТИЧКЕ ФУНКЦИЈЕ И ЦЕНА ПРЕЛАЗА	53
B.3.	БИНАРНИ ХИП СА АЖУРИРАЊЕМ ПРИОРИТЕТА	53
B.4.	МЕРЕЊЕ ЦЕНЕ ПУТА НА ЗАЈЕДНИЧКОЈ СКАЛИ	54
	ПРИЛОГ В: ИНСТРУМЕНТ ЗА КОРИСНИЧКО ТЕСТИРАЊЕ.....	55
V.1.	УВОДНА ИЗЈАВА И УПИТНИК О ПРЕТХОДНОМ ИСКУСТВУ	55
V.2.	ЗАДАЦИ.....	55
V.3.	УПИТНИК О УПОТРЕБЉИВОСТИ И ЗАВРШНА ПИТАЊА	56

1. Увод

Проналажење најкраћег пута између две тачке појављује се у рачунарству готово свуда. Планирање руте у навигацији, кретање ликова у играма, усмеравање пакета у мрежама и планирање путање мобилног робота различите су примене исте математичке основе, а то је претрага графа. Алгоритми ове врсте предају се на већини рачунарских студијских програма. Искуство у настави ипак показује да студенти савладају псеудокод и оцену сложености, а теже стичу интуицију о томе зашто се два поступка различито понашају на истом улазу.

Разлог лежи у природи материје. Разлика између претраге вођене пређеним путем и претраге вођене проценом преостале удаљености није очигледна из текста алгоритма, али постаје јасна чим се обе прикажу на истој мапи. Постојећи алати те врсте најчешће су настали као лични пројекти и задржали су се на приказу неколико алгоритама. Не нуде ни мерење ни упоредно покретање, па њихова употребљивост у систематској анализи остаје ограничена. Обимна метастудија показала је и да визуелизација сама по себи не побољшава исходе учења, већ да пресудан чинилац представља начин на који је ученик ангажован [17].

Циљ рада јесте пројектовање и реализација интерактивне веб апликације која повезује три целине које се у постојећим решењима ретко срећу заједно. Прва је визуелизација рада осам алгоритама на мрежном графу, уз потпуну контролу над извршавањем и уз приказ мерених величина. Друга је инфраструктура за поновљива мерења, која омогућава да се понашање испита на хиљадама аутоматски генерисаних мапа и да се резултати извезу за даљу обраду. Трећа је слој заснован на великим језичким моделима, чија је улога да објасни ток претраге, да генерише мапе према опису на природном језику и да препоручи алгоритам за задату поставку.

Доприноси рада су вишеструки. У практичном смислу реализована је потпуна апликација са клијентским и серверским делом, трајним чувањем података и подршком за два језика, што је чини употребљивом у настави. У истраживачком смислу спроведена је анализа на 9444 симулације, у којој су поред уобичајених величина измерени и меморијски отисак претраге и одступање пронађеног пута од доказано најјефтинијег. Уведена је заједничка скала цене пута која омогућава поређење поступака са различитим интерним дефиницијама растојања. Посебан допринос представља вредновање језичких модела у улози помоћног алата. У њему је показано да строго мерење тачности потцењује стварни учинак због честих изједначених резултата, па је предложена и примењена толерантна мера.

Рад је организован у десет поглавља. Друго поглавље излаже теоријске основе и описује разматране алгоритме и хеуристичке функције. Треће поглавље даје преглед постојећих решења, уочене недостатке и коришћене технологије. Четврто поглавље описује архитектуру и најважније делове реализације, а пето режиме рада и корисничко упутство. Шесто поглавље посвећено је интеграцији вештачке интелигенције. Седмо поглавље описује методологију мерења, осмо резултате вредновања алгоритама, а девето резултате вредновања вештачке интелигенције и корисничког интерфејса. У десетом поглављу сумирани су доприноси и назначени правци даљег рада.

2. АЛГОРИТМИ ЗА ПРОНАЛАЖЕЊЕ ПУТА НА МРЕЖНОМ ГРАФУ

Ово поглавље излаже теоријску подлогу рада. Најпре се уводи формална поставка проблема и објашњава превођење мреже ћелија у тежински граф. Затим се разматрају неинформисани поступци, поступци који узимају у обзир цену прелаза и поступци који користе процену преостале удаљености. Посебна пажња посвећена је породици изведеној пондерисањем хеуристике. Поглавље се завршава прегледом хеуристичких функција и упоредним приказом својстава свих разматраних алгоритама.

2.1. Формулација проблема

Проблем се дефинише над усмереним тежинским графом $G = (V, E, w)$. Скуп V чине чворови, скуп E чине гране, а функција w свакој грани придружује ненегативну цену прелаза. За задати почетни чвор s и циљни чвор t тражи се пут $P = (v_0, \dots, v_k)$ са $v_0 = s$ и $v_k = t$ чија је укупна цена најмања могућа.

$$c(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1}) \quad (2.1.1)$$

Уколико такав пут не постоји, алгоритам мора то поуздано да утврди и да се заустави. Ово наизглед споредно својство у пракси представља значајан извор грешака, због чега је посебно проверено на скупу мапа код којих је полазни чвор потпуно опкољен препрекама.

Граф овде није задат непосредно, већ настаје из дводимензионалне мреже ћелија. Свака ћелија представља чвор, а гране постоје између суседних ћелија које нису означене као препрека. Овакав приказ одговара мапама у играма, растерским мапама терена и заузетосним мрежама у роботизи, а у настави има додатну предност. Стање претраге приказује се непосредно на екрану.

Систем подржава два режима суседства. У режиму са четири суседа чвор је повезан са ћелијама изнад, испод, лево и десно, док се у режиму са осам суседа додају и дијагонални прелази. Цена прелаза одређена је тежином терена циљне ћелије, а код дијагоналних прелаза множи се чиниоцем $\sqrt{2}$, што одговара геометријском растојању између центара ћелија.

$$w(u, v) = \text{weight}(v) \cdot (\sqrt{2})^\delta, \quad \delta \in \{0, 1\} \quad (2.1.2)$$

Занимљива последица оваквог модела јесте да режим са осам суседа не даје само краће путеве, већ повећава и удео мапа на којима пут уопште постоји. Дијагонални прелаз може проћи између два зида који се додирују само у теменима. Ова појава квантитативно је потврђена у одељку 8.5.

2.2. Неинформисана претрага

Неинформисани поступци не располажу знањем о положају циља, већ систематски обилазе граф док на њега не наиђу.

Претрага у ширину, у литератури означена скраћеницом BFS, обилази граф слој по слој користећи ред [1]. Уколико све гране имају једнаку цену, први долазак до циља остварен

је путем са најмањим бројем корака. Границу те гаранције треба нагласити, јер се на том месту најчешће јавља неспоразум. Овај поступак минимизује број корака, а не укупну цену пута, па на мрежи са пондерисаним тереном најкраћи пут по броју корака не мора бити најјефтинији. Мерења изложена у поглављу 8 показују да просечно одступање износи 18,1 одсто, чиме теоријска напомена постаје мерљива. Временска сложеност износи $O(|V| + |E|)$, а просторна $O(|V|)$.

Претрага у дубину, означена скраћеницом DFS, разликује се само по структури података, тиме што уместо реда користи стек [1]. Иако сложеност остаје иста, понашање је потпуно другачије и не постоји никаква гаранција о квалитету пронађеног пута. У спроведеним мерењима просечно одступање износи 378,1 одсто, што значи да је пронађени пут скоро пет пута скупљи од оптималног. Са друге стране, поступак понекад пронађе циљ уз врло мали број проширених чворова, и то када смер претраге случајно води ка циљу. Због те непредвидивости он има изразиту наставну вредност. Непосредно показује да мали број проширених чворова сам по себи не значи добар резултат.

2.3. Претрага са ценом прелаза

Када гране немају једнаку цену, редослед обраде мора да зависи од укупне цене пута од полазног чвора.

Дајкстрин алгоритам, објављен 1959. године [3], у сваком кораку бира чвор са најмањом до сада утврђеном ценом и проглашава је коначном. Исправност почива на ненегативности цена грана, због чега ниједан касније пронађен пут не може бити јефтинији од већ утврђеног. Поступак гарантује најјефтинији пут и у овом раду служи као референца у односу на коју се мери одступање осталих алгоритама. Цена те гаранције огледа се у обиму претраге, пошто се frontiјер шири подједнако у свим смеровима. Уз приоритетни ред реализован као бинарни хип сложеност износи $O((|V| + |E|) \log |V|)$ [1].

За посебан случај у коме све гране имају цену 0 или 1 постоји поступак који даје оптималан резултат без приоритетног реда. Уместо њега користи се двострани ред, у који се чвор достигнут граном цене 0 додаје на почетак, а чвор достигнут граном цене 1 на крај [10]. Тиме се уређење по цени одржава уз сложеност $O(|V| + |E|)$. Идеја припада широј породици поступака који приоритетни ред замењују распоредом по познатим вредностима цене [11]. Овај поступак укључен је због наставне вредности, јер показује како познавање структуре улаза омогућава специјализацију општег решења. Његово укључивање у мерења донело је и налаз који није био очекиван, о чему се расправља у одељку 8.8.

2.4. Информисана претрага

Информисани поступци користе хеуристичку функцију $h(n)$, која процењује преосталу удаљеност до циља и омогућава да се претрага усмери.

Алгоритам A^* , представљен 1968. године [4], бира чвор са најмањом процењеном укупном ценом пута кроз тај чвор.

$$f(n) = g(n) + h(n) \quad (2.4.1)$$

Величина $g(n)$ представља утврђену цену пута од полазног чвора, а $h(n)$ процену цене преосталог дела. Уколико хеуристика никада не прецењује стварну преосталу цену, за њу се каже да је допустива, и тада поступак гарантује најјефтинији пут [4]. Уколико поред тога важи и услов доследности, ниједан чвор неће бити обрађен више пута, чиме поступак постаје и ефикаснији [5, 7]. Алгоритам A^* може се посматрати као уопштење претходних поступака.

За $h(n) = 0$ своди се на Дајкстрин алгоритам. Са растом утицаја хеуристике постаје све сличнији поступку који узима у обзир само процену [12].

Похлепна претрага са најбољим првим избором користи само хеуристику и потпуно занемарује пређени пут, то јест $f(n) = h(n)$ [2]. Тиме брзо напредује ка циљу, али губи сваку гаранцију оптималности. У отвореном простору понаша се готово идеално, док у околини препрека често уђе у слепу улицу. Мерења показују да у просеку обради најмањи број чворова од свих разматраних поступака. Њен пут ипак одступа од најјефтинијег за 23,9 одсто. Ова два налаза заједно чине најјаснију илустрацију компромиса између цене претраге и квалитета решења.

2.5. Породица пондерисаних варијанти

Између алгоритма A^* и похлепне претраге постоји низ међурешења која настају увођењем тежинског чиниоца испред хеуристике [6, 9].

$$f(n) = g(n) + w \cdot h(n), \quad w \geq 1 \quad (2.5.1)$$

За $w = 1$ добија се алгоритам A^* уз све његове гаранције. За веће вредности хеуристика добија већи утицај, претрага се брже усмерава ка циљу и број проширених чворова опада, док се оптималност губи, али не неограничено. Познат резултат каже да цена пронађеног пута није већа од w пута цене најјефтинијег пута, што овај поступак чини ограничено субоптималним [6, 9]. Ова особина искоришћена је у поступцима са временским ограничењем, попут алгоритма ARA^* , који постепено смањује вредност w и тако побољшава решење док траје расположиво време [8].

Реализоване су две подешене варијанте, уз терминологију преузету из познатог визуелизатора [22]. Варијанта *Swarm* користи $w = 2$, а *Convergent Swarm* $w = 5$. Њихове реализације не разликују се ни у једном другом детаљу, што је експериментално потврђено у одељку 8.6, где за исто задато w производе идентичне резултате.

2.6. Хеуристичке функције

Систем подржава четири хеуристике, чије су дефиниције дате у наставку, где су Δr и Δc апсолутне разлике редова и колона између посматраног чвора и циља.

$$h_M = \Delta r + \Delta c \quad (2.6.1)$$

$$h_E = \sqrt{(\Delta r)^2 + (\Delta c)^2} \quad (2.6.2)$$

$$h_C = \max(\Delta r, \Delta c) \quad (2.6.3)$$

$$h_O = (\sqrt{2} - 1) \cdot \min(\Delta r, \Delta c) + \max(\Delta r, \Delta c) \quad (2.6.4)$$

Њихова својства сажета су у табели 2.6.1. Манхетн растојање представља тачно растојање у режиму са четири суседа, због чега је у том режиму истовремено допустиво и максимално информативно. Еуклидско растојање увек потцењује стварно растојање на мрежи, па остаје допустиво, али слабије усмерава претрагу. Чебишевљево растојање одговара мрежи у којој дијагонални корак кошта исто колико и ортогонални, док је октилно растојање прилагођено режиму са осам суседа и у њему представља најпрецизнију процену.

Табела 2.6.1. Својства хеуристичких функција

Хеуристика	Израз	Допустива за 4 суседа	Допустива за 8 суседа	Најпогоднија примена
Манхетн	$\Delta r + \Delta c$	да, тачна	не, прецењује	мреже са 4 суседа

Хеуристика	Израз	Допустива за 4 суседа	Допустива за 8 суседа	Најпогоднија примена
еуклидска	$\sqrt{(\Delta r^2 + \Delta c^2)}$	да, потцењује	да, потцењује	опште мреже
Чебишевљева	$\max(\Delta r, \Delta c)$	да, потцењује	да, потцењује	једнаки кораци
октилна	$(\sqrt{2}-1) \cdot \min + \max$	да, потцењује	да, тачна	мреже са 8 суседа

Треба нагласити да Манхетн растојање у режиму са осам суседа прецењује стварну удаљеност, чиме престаје да буде допустиво и алгоритам A^* формално губи гаранцију оптималности. Ова методолошка напомена битна је при тумачењу резултата из одељка 8.4.

2.7. Упоредни преглед својстава

Табела 2.7.1 сажима основне карактеристике свих осам разматраних алгоритама. Условна оптималност означава да гаранција важи само уз испуњење наведене претпоставке.

Табела 2.7.1. Упоредни преглед разматраних алгоритама

Алгоритам	Приоритетна функција	Структура података	Оптималност	Услов оптималности
BFS	нема	ред	условна	све гране имају једнаку цену
DFS	нема	стек	нема	није примењиво
Dijkstra	$g(n)$	бинарни хип	потпуна	ненегативне цене грана
A^*	$g(n) + h(n)$	бинарни хип	условна	допустива хеуристика
Greedy	$h(n)$	бинарни хип	нема	није примењиво
Swarm	$g(n) + 2 \cdot h(n)$	бинарни хип	ограничена	цена $\leq 2 \times$ оптимална
Conv. Swarm	$g(n) + 5 \cdot h(n)$	бинарни хип	ограничена	цена $\leq 5 \times$ оптимална
0-1 BFS	нема	двострани ред	условна	цене грана из скупа $\{0, 1\}$

Из табеле се уочава да пет од осам поступака деле исти начин рада и разликују се искључиво по приоритетној функцији. То су Дајкстрин алгоритам, алгоритам A^* , похлепна претрага и обе пондерисане варијанте. Ова чињеница искоришћена је у имплементацији, где су сви наведени поступци реализовани кроз јединствену машину претраге, што је описано у одељку 4.3. Поред разматраних поступака, литература познаје и напреднија решења прилагођена мрежним мапама, попут алгоритма Jump Point Search [13], поступка Theta* [14] и алгоритма D* Lite за динамичка окружења [15]. Они нису укључени у систем, а разлози су образложени у одељку 10.2.

Ово поглавље поставило је теоријски оквир рада и показало да свака гаранција оптималности важи само унутар претпоставки под којима је изведена, што ће се у експерименталном делу показати као мерљива појава. Наредно поглавље разматра како су ове идеје обрађене у постојећим решењима.

3. ПРЕГЛЕД ПОСТОЈЕЋИХ РЕШЕЊА И КОРИШЋЕНЕ ТЕХНОЛОГИЈЕ

Ово поглавље сагледава шта у овој области већ постоји и где су границе тих решења, а затим наводи технологије изабране за реализацију сопственог система. Разматрају се три карактеристична представника, изабрана тако да покрију распон од библиотеке намењене програмерима, преко наставног визуелизатора, до свеобухватне платформе за учење структура података. На основу уочених недостатака изводе се захтеви који су обликовали систем описан у наредним поглављима.

3.1. Постојећа решења

Замисао да се рад алгоритма прикаже анимацијом није нова. Први системи те врсте настали су још осамдесетих година прошлог века [20]. Тек је појава веб технологија омогућила да такви алати постану широко доступни.

Најпознатије решење у овој области јесте визуелизатор који је објавио Клеман Михаилеску [22]. Реч је о веб апликацији написаној искључиво помоћу језика JavaScript, без развојног оквира и без серверског дела, која подржава осам алгоритама и генератор лавирината заснован на рекурзивној подели простора. Управо ово решење увело је терминологију коју користи и систем описан у овом раду. Аутор наводи да је поступак Swarm осмислио заједно са колегом, међутим из формулације приоритетне функције види се да је реч о познатом пондерисаном A^* претраживању, описаном још 1970. године [6]. Ова околност показује да су у практичној заједници називи понекад настајали независно од академске терминологије, што код студената може створити утисак да је реч о новим поступцима. Главна ограничења произлазе из природе личног пројекта: апликација не памти стање, не омогућава упоредно покретање више алгоритама над истом мапом, не приказује нумеричке показатеље и не нуди извоз резултата. Последњи допринос изворном коду забележен је 2016. године.

Библиотека PathFinding.js, чији је аутор Сјуећао Сју, представља друкчију врсту решења [23]. Реч је о библиотеци намењеној уградњи у веб игре, која подржава једанаест поступака, међу којима су и Jump Point Search и двосмерне варијанте, уз четири хеуристичке функције и подешавање дијагоналног кретања. Уз њу долази демонстрациона страница, али је она замишљена као допуна документацији, а не као наставно средство. Централно ограничење у контексту овог рада јесте то што библиотека познаје само проходне и непроходне ћелије. Пондерисан терен зато није подржан. Разлика између Дајкстриног алгоритма и претраге у ширину практично се не може демонстрирати. Последње објављено издање датира из 2014. године.

Платформа VisuAlgo, коју од 2011. године развија тим предвођен Стивенем Халимом на Националном универзитету у Сингапуру, представља најозбиљније наставно решење [21]. Обухвата преко двадесет модула, међу којима су и модули за обилазак графа и за најкраће путеве од једног извора. Поседује и режим за вежбање у коме систем аутоматски генерише питања и оцењује одговоре. Ограничење у односу на потребе овог рада произлази из њене

опште намене, зато што се графови приказују апстрактно, а не као мрежа ћелија која представља мапу. Због тога изостају појмови суштински за проналажење пута на мапама, попут густине препрека, пондерисаног терена и избора суседства, а алгоритам A^* и породица пондерисаних варијанти нису обухваћени.

3.2. Упоредна анализа и изведени захтеви

Табела 3.2.1 сажима својства разматраних решења и поставља их наспрам система описаног у овом раду. Ознака делимичне подршке значи да својство постоји, али у обиму који не задовољава потребе систематске анализе.

Табела 3.2.1. Упоредни преглед постојећих решења

Својство	Михаилеску [22]	PathFinding.js [23]	VisuAlgo [21]	Овај рад
Број алгоритама	8	11	3 за најкраћи пут	8
Мрежни приказ мапе	да	да	не	да
Пондерисан терен	не	не	да, апстрактно	да
Избор хеуристике и суседства	не	да	није примењиво	да
Поређење више алгоритама	не	не	делимично	да
Приказ мерених величина	не	не	делимично	да
Извоз резултата	не	не	не	да
Аутоматско генерисање мапа	делимично	не	да	да
Трајно чување корисничких мапа	не	не	делимично	да
Провера знања	не	не	да	да
Подршка вештачке интелигенције	не	не	не	да
Активан развој	не	не	да	да

Из прегледа се уочавају три празнине које су обликовале захтеве постављене пред систем. Прва се односи на мерење, пошто ниједно решење не приказује нумеричке показатеље током рада нити омогућава њихов извоз, због чега корисник не може да поткрепи закључак стечен посматрањем. Друга се односи на поређење, јер ниједно решење не омогућава истовремено покретање више поступака над истом мапом, што управо представља најдиректнији начин да се уоче њихове разлике. Трећа се односи на активну улогу корисника. Сва решења осим платформе VisuAlgo корисника остављају у улози посматрача.

Литература о ефикасности визуелизације пружа снажан аргумент у прилог трећој празнини, зато што је показано да пресудан чинилац представља начин на који је ученик ангажован [17]. Исти закључак разрађен је кроз таксономију нивоа ангажовања [18]. Она разликује посматрање, реаговање, мењање, конструисање и представљање. Каснији преглед стања области указао је на то да многи алати остају неупотребљени у настави управо зато што не предвиђају активност ученика [19]. Ови налази непосредно су обликовали два пројектна решења. Прво је режим полигона, у коме корисник сам конструира пут и добија оцену. Друго је укључивање мерених величина у сам кориснички интерфејс, чиме се посматрање допуњује бројчаном потврдом.

3.3. Коришћене технологије

Систем је реализован као веб апликација са раздвојеним клијентским и серверским делом. Табела 3.3.1 наводи изабране технологије уз верзије којима је систем изграђен и уз разлог за сваки избор.

Табела 3.3.1. Преглед технолошких избора и разлога

Слој	Изабрано решење	Основни разлог
Кориснички интерфејс	Angular 21.2 [35]	реактивно управљање стањем анимације
Програмски језик	TypeScript 5.9	заједнички типови на клијенту и серверу
Приказ мреже	HTML5 Canvas	приказ до 20000 ћелија у једном пролазу
Обликовање	Tailwind CSS 4.2 [39]	доследан изглед без засебних стилских датотека
Серверски оквир	Express 5.2 [36] на Node.js 24.12	исти језик као на клијенту, поновна употреба алгоритама
База података	MongoDB уз Mongoose 9.4 [37]	документни модел одговара мапама и траговима
Провера улаза	Zod 4.3	јединствен извор за проверу и за типове
Стварно време	Socket.io 4.8 [38]	ажурирање табеле резултата без освежавања странице
Аутентикација	JSON веб токени и bcrypt	стандардан приступ без чувања стања на серверу
Језички модели	GitHub Models [40]	замена модела изменом једне поставке
Обрада мерења	Python 3.14	зрели алати за статистику и графику

Избор развојног оквира Angular заснован је на три особине које одговарају природи система. Прва је уграђена подршка за реактивно програмирање помоћу библиотеке RxJS, што знатно олакшава управљање стањем анимације која се мења много пута у секунди. Друга је систем самосталних компонената, који уклања потребу за модулима и чини структуру пројекта прегледнијом. Трећа је потпуна интеграција са језиком TypeScript. Захваљујући њој типови који описују мрежу, догађаје претраге и резултате смештени су у заједнички директоријум и користе се и на клијенту и на серверу. Тиме је искључена читава класа грешака у којој се структуре података на две стране разиђу.

Приказ мреже реализован је помоћу елемента HTML5 Canvas. Алтернатива, то јест приказ сваке ћелије као засебног елемента документа, била би непрактична. Мрежа од сто редова и две стотине колона садржи двадесет хиљада ћелија. Ниједан прегледач их не би приказивао довољно брзо.

Избор платформе Node.js на серверу произлази из чињенице да сервер и клијент користе исти програмски језик, што омогућава да се једном написани алгоритми покрећу на обе стране система. Ова могућност искоришћена је најизраженије у компоненти вештачке интелигенције, у оквиру које сервер покреће све алгоритме над кандидат мапама пре него што упути захтев језичком моделу.

Сервис GitHub Models изабран је из два разлога. Први је практичне природе, јер је у оквиру академске лиценце доступан без надокнаде. Други се показао значајнијим. Једно исто

сучеље омогућава замену модела изменом једне поставке, па је постало изводљиво упоредити више модела на истом скупу задатака. То је искоришћено у вредновању изложеном у одељку 9.1.

Развој је обављен у окружењу Visual Studio Code, уз алате `ts-node` и `nodemon` за покретање серверског кода и `Angular CLI` за изградњу клијентског дела. Статистичка обрада измерених података и израда графичких приказа обављени су у језику Python уз библиотеке `pandas`, `NumPy`, `SciPy` и `Matplotlib`.

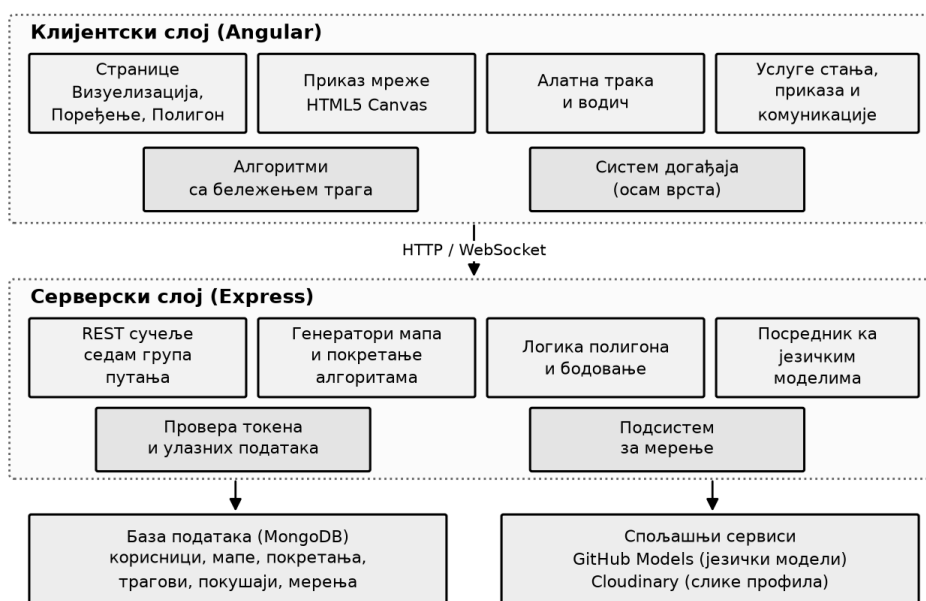
Ово поглавље показало је да постојећа решења покривају приказ рада алгоритама, али да заједно не покривају мерење, поређење и активно учешће корисника, и навело је технолошку основу изабрану за реализацију. Наредно поглавље прелази на архитектуру система.

4. АРХИТЕКТУРА И ИМПЛЕМЕНТАЦИЈА СИСТЕМА

Ово поглавље описује унутрашњу грађу система. Најпре се излаже општа архитектура и подела одговорности. Затим се разматра јединствени интерфејс алгорита са системом догађаја, који чини језгро решења. Потом следе имплементација алгоритама, приказ мреже, управљање репродукцијом, генератори мапа, серверски слој и безбедносни механизми.

4.1. Општа архитектура

Систем је организован у три слоја, приказана на слици 4.1.1. Клијентски слој обухвата кориснички интерфејс, извршавање алгоритама и приказ на платну. Серверски слој обухвата програмски интерфејс, пословну логику полигона, генераторе мапа и посреднички слој ка језичким моделима. Трећи слој чини база података, у којој се чувају корисници, мапе, покретања, трагови извршавања и покушаји у режиму полигона. Потпуна структура директоријума изложена је у прилогу А.



Слика 4.1.1. Слојевита архитектура система

Значајна пројектна одлука односи се на место извршавања алгоритама. Они се извршавају и на клијенту и на серверу, али са различитим наменама. На клијенту се извршавају ради приказа и сваки корак се бележи као догађај. На серверу се извршавају без бележења трага, ради брзине. Тако се обављају израчунавање референтног решења у режиму полигона, провера кандидат мапа и покретање мерења. Клијентска реализација оптимизирана је за прегледност трага, а серверска за пропусност, зато што обради више од две хиљаде симулација у секунди.

4.2. Јединствени интерфејс алгорита

Централна пројектна идеја јесте потпуно раздвајање алгорита од приказа. Приказ не зна ништа о томе који се алгоритам извршава. Он реагује искључиво на низ догађаја које алгоритам емитије. Због тога додавање новог алгорита не захтева ниједну измену у делу задуженом за приказ.

Сваки алгоритам реализује исти интерфејс, наведен у прилогу Б. Њега чине метода за иницијализацију, метода за корак, метода за проверу завршетка и метода за резултат. Метода за корак извршава једну логичку операцију и враћа списак насталих догађаја. Реч је о примени обрасца државне машине, пошто алгоритам није написан као једна петља која се извршава до краја, већ је разложен на дискретне кораке које позивалац покреће када му одговара. Тиме се анимација може паузирати између било која два корака, корисник може прећи на произвољан корак, а исти код може се покренути и без приказа.

Догађаји представљају јединствен језик којим сви алгоритми описују шта раде. Дефинисано је осам врста, наведених у табели 4.2.1. Пошто сваки догађај описује промену, а не цело стање, могуће је кретати се уназад применом обрнуте операције, што је искоришћено за дугме за корак уназад које у постојећим решењима не постоји.

Табела 4.2.1. Врсте догађаја које емитију алгоритми

Догађај	Значење	Емитије се
INIT	почетак претраге, задате поставке	једном, на почетку
SET_CURRENT	чвор који се тренутно обрађује	у сваком кораку
OPEN_ADD	чвор додат у фронтијер	при откривању новог чвора
OPEN_UPDATE	побољшана цена чвора у фронтијеру	при проналажењу јефтинијег пута
CLOSE_ADD	чвор пребачен у скуп обрађених	након проширења чвора
RELAX_EDGE	опуштање гране, промена родитеља	при ажурирању цене
FOUND_PATH	пронађен пут и његова укупна цена	једном, при успеху
NO_PATH	фронтијер је исцрпљен без успеха	једном, при неуспеху

4.3. Имплементација алгорита

Реализација прати запажање из одељка 2.7 да пет од осам поступака деле исти начин рада. Због тога су Дајкстрин алгоритам, алгоритам А*, похлепна претрага и обе пондерисане варијанте реализовани кроз једну класу, названу машина претраге са најбољим првим избором. Разлика међу њима сведена је на једну методу која рачуна приоритет.

```
private computeF(g: number, h: number): number {
  switch (this.options.algorithm) {
    case AlgorithmType.DIJKSTRA:      return g;
    case AlgorithmType.GREEDY:        return h;
    case AlgorithmType.A_STAR:         return g + h;
    case AlgorithmType.SWARM:          return g + (this.options.swarmWeight ?? 2) * h;
    case AlgorithmType.CONVERGENT_SWARM: return g + (this.options.swarmWeight ?? 5) * h;
    default:                          return g + h;
  }
}
```

Практична вредност овог решења показала се током исправљања грешака. Свака исправка у машини претраге истовремено обухвата пет алгорита, а разлике у понашању

могу да потичу само из приоритетне функције. Простор за тражење узрока грешке тиме је знатно сужен.

Приоритетни ред реализован је као бинарни хип који подржава и промену приоритета већ уписаног чвора, а његова реализација изложена је у прилогу Б. Претрага у ширину и претрага у дубину реализоване су засебно, а претрага 0-1 у ширину користи двострани ред. Код последњег поступка уочен је детаљ који није очигледан из псеудокода, пошто двострани ред може да садржи више уноса за исти чвор, а каснији су застарели. Уколико се такви уноси не одбаце, приказ би исти чвор обележио више пута, а бројач проширених чворова дао би превисоку вредност, па је проблем решен увођењем скупа већ обрађених чворова.

4.4. Приказ мреже и управљање репродукцијом

Приказ је реализован као засебна услуга која на основу текућег стања исцртава целу мрежу, а свака ћелија добија боју према улози коју у том тренутку има. Приликом реализације решена су два проблема која нису очигледна унапред. Први се односи на екране са повећаном густином тачака, на којима платно подразумевано изгледа замућено. Решење је да се физичка величина платна и цртачки контекст скалирају чиниоцем густине приказа. Уз то, претварање положаја показивача у координате ћелије мора да користи логичке пикселе. Други се односи на променљиве димензије мреже, пошто систем допушта од пет до две стотине редова и колона. Решење је да укупна површина приказа остане непроменљива, а да се величина ћелије израчуна дељењем расположиве површине бројем ћелија.

Управљање током анимације поверено је услузи која одржава државну машину са четири стања: мировање, извршавање, пауза и завршетак. Пре почетка анимације алгоритам се изврши до краја и цео траг се сачува, чиме притисак на дугме за покретање даје тренутан одзив и омогућава се клизач за премотавање.

4.5. Генератори мапа

Ручно цртање мапа погодно је за истраживање, али не и за систематско мерење, где је потребан велики број различитих, а поновљивих улаза. Реализовано је седам генератора, наведених у табели 4.5.1.

Табела 4.5.1. Реализовани генератори мапа

Генератор	Опис	Сврха у мерењу
отворено поље	мрежа без иједне препреке	основна референца, чист утицај хеуристике
случајне препреке	препреке распоређене случајно уз задату густину	утицај густине препрека
лабиринт	рекурзивна подела простора уз гарантовану проходност	дуги вијугави путеви
пондерисан терен	зоне са повећаном тежином прелаза	разлика између броја корака и цене
уско грло	велики простор са једним уским пролазом	понашање хеуристике при заводљивом смеру
градски блокови	правилни ортогонални коридори	погодност Манхетн хеуристике
мешовит	препреке и пондерисан терен заједно	најсложенији случај

Сваки генератор прима зрно случајног низа, чиме је обезбеђена поновљивост. Исто зрно увек производи исту мапу. Генератор псеудослучајних бројева реализован је као

линеарни конгруентни генератор са непроменљивим параметрима, независно од уграђеног генератора платформе, чија реализација може да се разликује између издања. Генератор лавирината ради рекурзивном поделом простора, тако што у сваком кораку постави зид и у њему остави један отвор, због чега је добијени лавиринт по конструкцији проходан.

4.6. Серверски слој, модел података и безбедност

Серверски слој излаже програмски интерфејс подељен у седам група путања: аутентикација, мапе, покретања, полигон, вештачка интелигенција, мерење и слање слика. Свака путања која приступа корисничким подацима заштићена је посредником који проверава веб токен. Свака путања која прима идентификатор проверава да ли је он исправног облика пре него што приступи бази. Путање које примају мрежу проверавају да ли су координате полазног и циљног чвора унутар задатих димензија. Изостанак те провере доводи до приступа изван граница низа.

База података садржи шест колекција, наведених у табели 4.6.1. Брисање мапе покреће брисање свих зависних записа, односно покретања, трагова и покушаја у режиму полигона. Ова каскада уведена је накнадно, након што је уочено да брисање мапе оставља записе који упућују на непостојећи документ.

Табела 4.6.1. Колекције у бази података

Колекција	Садржај	Веза
users	налози, сажетак лозинке, адреса слике профила	нема
maps	мапе, зидови, тежине, полазни и циљни чвор	власник је корисник
runs	покретања алгоритама са мереним величинама	мапа и корисник
traces	низови догађаја за свако покретање	покретање
playgroundattempts	покушаји корисника, поени и разлагање поена	мапа и корисник
benchmarkresults	појединачни резултати мерења	груписано по ознаци пакета

Аутентикација је заснована на веб токенима. Приликом пријаве сервер проверава лозинку поређењем са сачуваним сажетком добијеним функцијом `bcrypt` и издаје потписан токен који клијент чува локално. Уграђено је више заштитних мера. Број покушаја пријаве ограничен је на десет у периоду од петнаест минута по адреси. Безбедносна заглавља поставља посредник `helmet`, а величина тела захтева ограничена је на два мегабајта. Клијент аутоматски одјављује корисника након петнаест минута неактивности. Приступ туђим подацима спречен је провером власништва на свакој путањи, а кључ за приступ језичким моделима никада не напушта сервер. Систем подржава српски и енглески језик, као и светлу и тамну тему. Боје које се цртају на платну услуга за приказ бира према текућој теми, јер се оне не могу дефинисати као променљиве стилских листова.

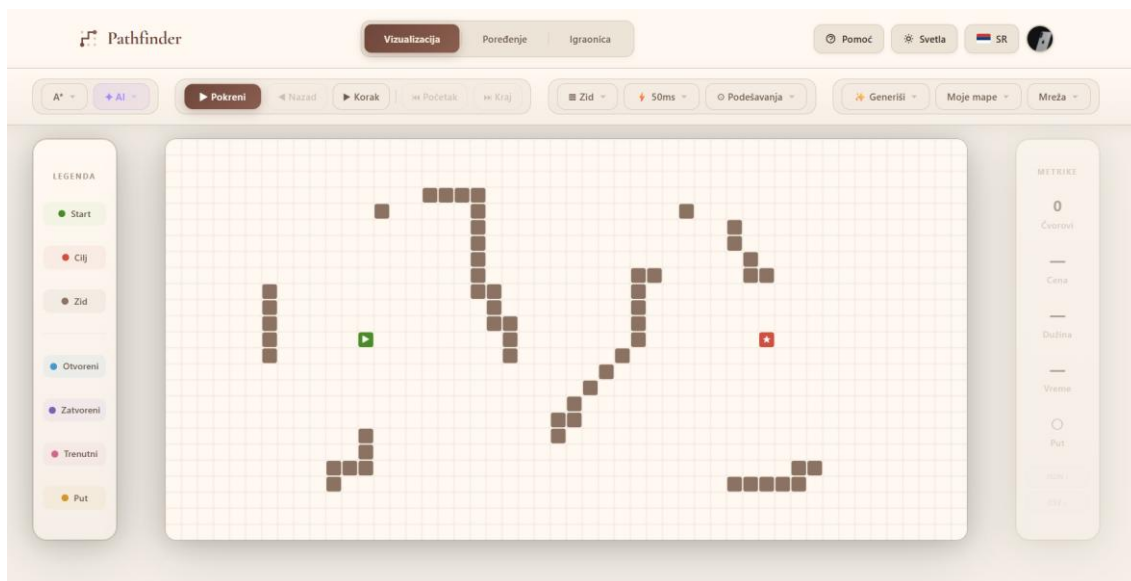
Ово поглавље описало је грађу система. Централно место у њој заузима јединствен интерфејс алгорита са системом догађаја, који је омогућио да приказ остане потпуно независан од алгоритама. Наредно поглавље приказује како се ове могућности испољавају у режимима рада доступним кориснику.

5. РЕЖИМИ РАДА И КОРИСНИЧКО УПУТСТВО

Систем нуди три главна режима рада, којима се приступа преко стално видљиве навигације. Ово поглавље описује сваки од њих и уједно служи као корисничко упутство, јер наводи шта поједина функција ради и какав низ радњи треба извести да би се остварио одређени сценарио. На крају поглавља описани су интерактивни водич и рад са корисничким налогом.

5.1. Заједнички распоред екрана

Сва три режима деле исти распоред, што је свесна одлука донета ради лакшег учења интерфејса. Горњи део екрана заузима алатна трака подељена у четири логичке групе: избор алгоритма и приступ модулима вештачке интелигенције, управљање репродукцијом, алати за цртање са подешавањима, и рад са мапама. Испод алатне траке налази се централни део, у коме је мрежа окружена двема бочним површинама. Лева садржи легенду која објашњава значење боја, а десна приказује мерене величине. Обе површине стално су видљиве и прате висину мреже. Тиме је остварен принцип препознавања уместо присећања. Корисник не мора да памти шта која боја значи нити да отвара засебан прозор да би видео резултат. Слика 5.1.1 приказује овај распоред.

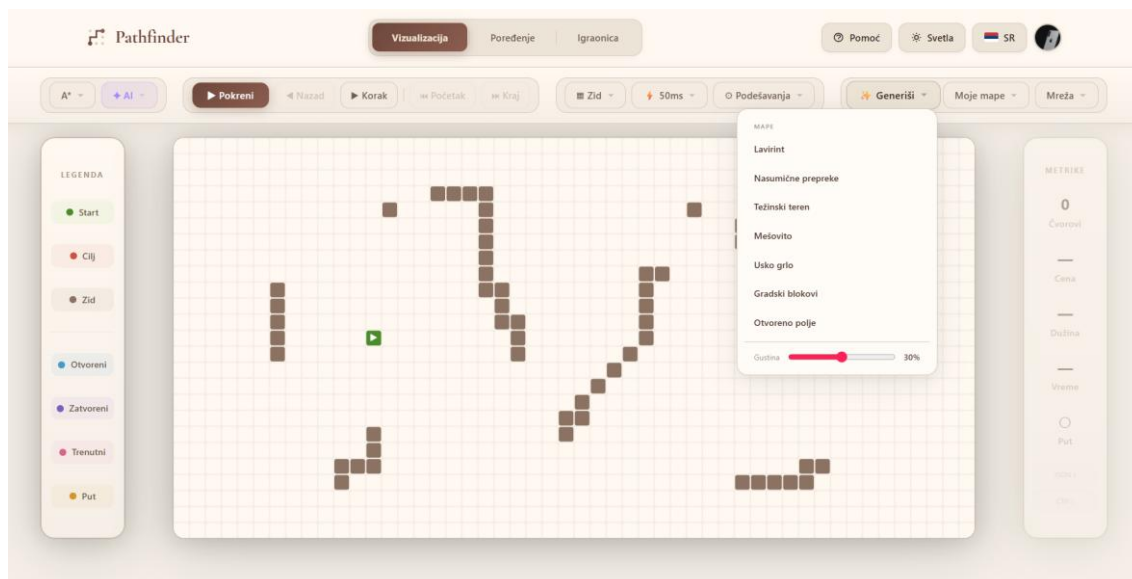


Слика 5.1.1. Заједнички распоред екрана у свим режимима

5.2. Режим визуелизације

Режим визуелизације представља основни начин рада. Корисник најпре обликује мапу, било ручно било помоћу генератора, затим бира алгоритам и његове поставке, и на крају покрене анимацију.

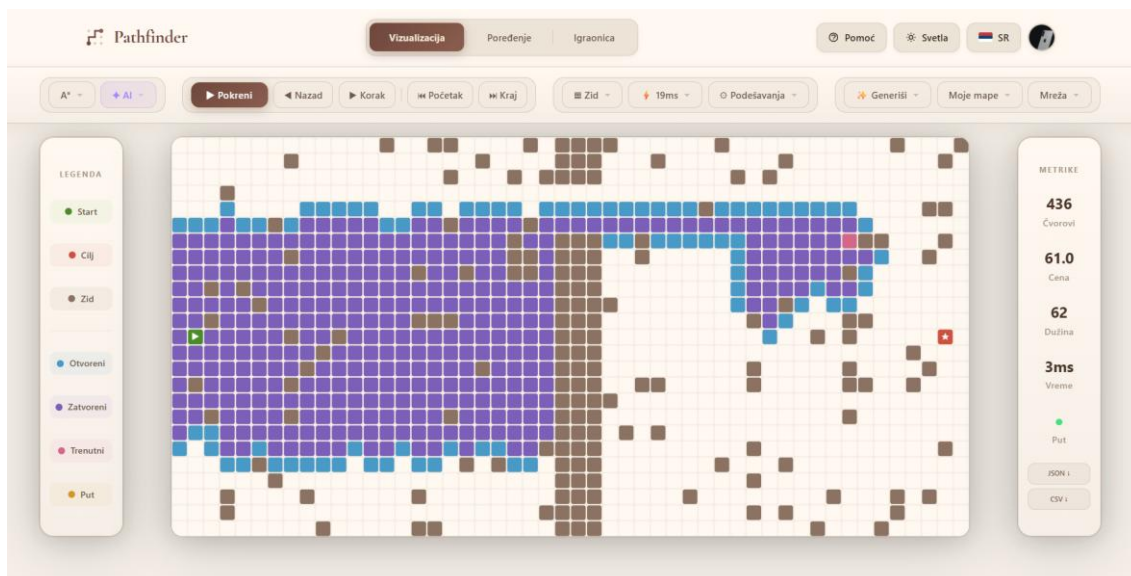
Уређивање мапе обавља се показивачем. Превлачењем се цртају зидови, десним тастером се бришу, а полазни и циљни чвор премештају се превлачењем. Алат за пондерисан терен омогућава задавање тежине у распону од два до десет, а задата вредност приказује се у самој ћелији. Систем не дозвољава постављање зида на полазни или циљни чвор, чиме се спречава ситуација у којој претрага нема одакле да почне. Уместо ручног цртања могуће је употребити и неки од седам генератора описаних у одељку 4.5, што је приказано на слици 5.2.1.



Слика 5.2.1. Аутоматско генерисање мапе

Поставке алгоритма обухватају избор хеуристике, избор суседства и вредност тежинског параметра за пондерисане варијанте. Контроле које нису примењиве на изабрани алгоритам аутоматски се онемогућавају, па се тако при избору претраге у ширину избор хеуристике затамњује, чиме се спречава да корисник подеси вредност која нема утицаја.

Током анимације десна површина приказује број проширених чворова, цену пута, дужину пута у корацима и време извршавања, а вредности се ажурирају док анимација траје. Управљање обухвата покретање, паузу, кораке напред и назад, прелазак на почетак и крај, као и подешавање брзине у распону од десет до две стотине милисекунди по кораку. Слика 5.2.2 приказује стање мреже док је анимација заустављена усред извршавања алгоритма A*, на којој се јасно разликују frontiјер, обрађени чворови и чвор који се тренутно обрађује. Мерене величине у том тренутку већ приказују коначне вредности, зато што се траг израчунава пре почетка анимације, што је објашњено у одељку 4.4. По завршетку је могуће извести резултат у формату JSON или CSV.



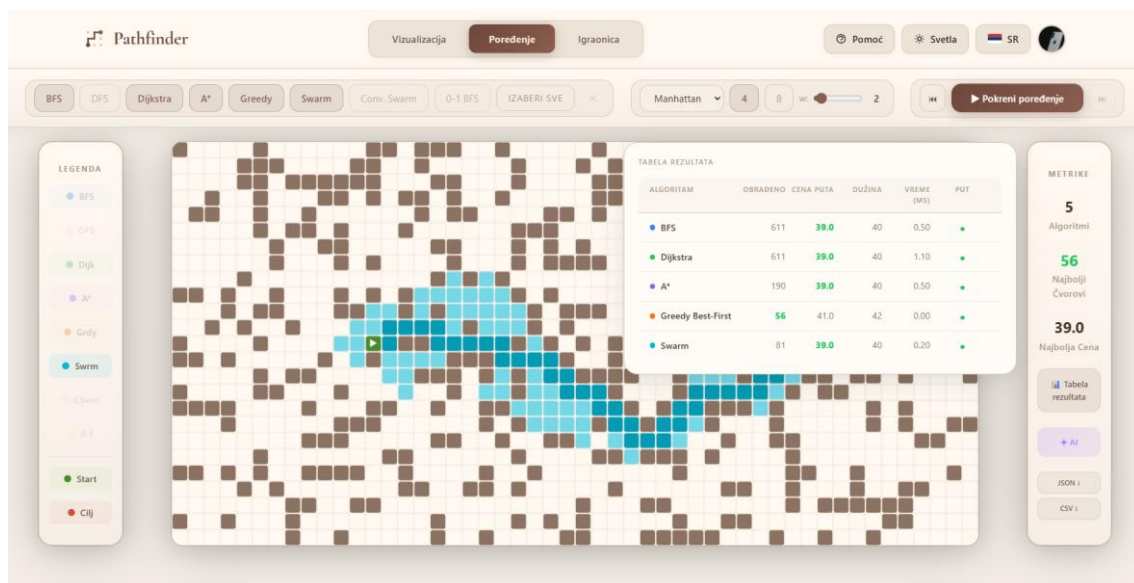
Слика 5.2.2. Ток претраге у режиму визуелизације

Типичан сценарио у настави изгледа овако. Наставник изабере генератор пондерисаног терена, покрене претрагу у ширину и запамти добијену цену пута, затим покрене Дајкстрин алгоритам на истој мапи и упореди вредности. Разлика која се тако добије непосредно показује да претрага у ширину минимизује број корака, а не цену.

5.3. Режим поређења

Режим поређења одговара на питање како се алгоритми понашају један према другом на потпуно истом улазу. Корисник бира произвољан подскуп од осам алгоритама, а предвиђена су и дугмад за избор свих и за поништавање избора, након чега систем покреће све изабране поступке над истом мапом.

Резултати се приказују у табели која садржи број проширених чворова, цену пута, дужину пута и време извршавања за сваки алгоритам, а најбоља вредност у свакој колони посебно је означена. Кликом на ред табеле приказује се пут који је тај алгоритам пронашао, чиме се разлика види непосредно на мапи. Свакој ставци додељена је непроменљива боја, иста на све три странице, тако да корисник једном научено мапирање боје и алгоритма користи свуда. Изглед екрана приказан је на слици 5.3.1.



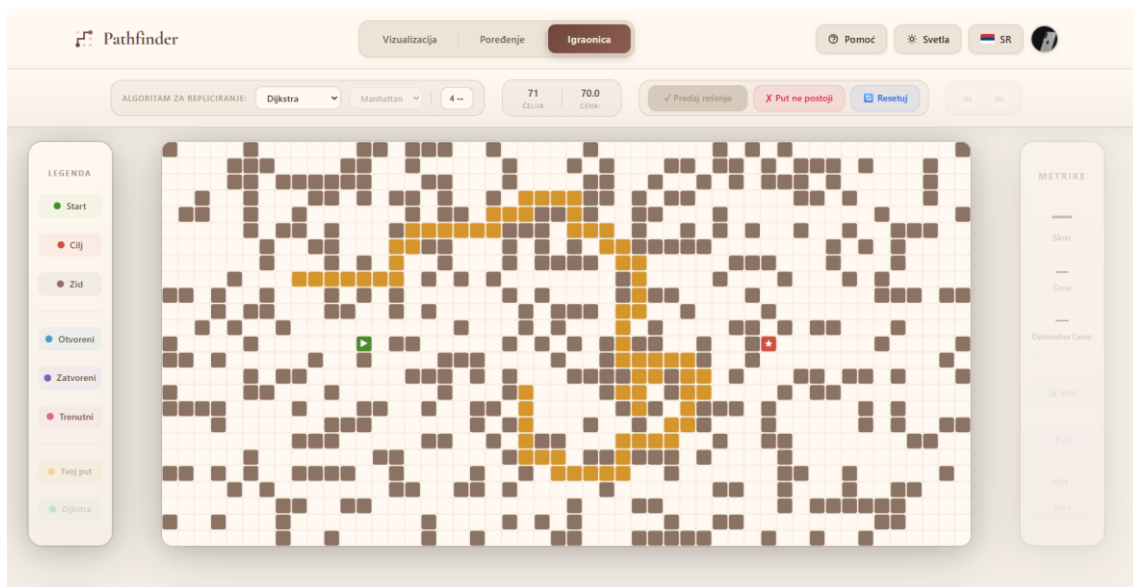
Слика 5.3.1. Упоредни приказ рада више алгоритама

У овом режиму трагови се не бележе, пошто се анимација не приказује. Време извршавања свих осам поступака на мрежи стандардних димензија тако се своди на неколико милсекунди. То омогућава брзо понављање експеримента са измењеном мапом.

5.4. Режим полигона

Режим полигона представља одговор на налаз изложен у одељку 3.2, према коме пасивно посматрање даје слабе исходе учења. У овом режиму корисник не посматра рад алгорита, већ сам покушава да конструише пут.

Ток рада тече на следећи начин. Корисник бира референтни алгоритам чије решење покушава да достигне. Затим кликовима гради пут од полазног до циљног чвора, а десним кликом уклања последњи додати корак. Систем проверава да ли је свака нова ћелија суседна претходној и да ли је проходна. Уколико закључи да пут не постоји, уместо конструисања пута бира одговарајућу опцију. Ова фаза приказана је на слици 5.4.1.



Слика 5.4.1. Конструисање пута у режиму полигона

Након потврде, сервер покреће изабрани алгоритам и враћа референтно решење. Кључно је да се то израчунавање обавља на серверу, чиме је онемогућено да корисник унапред види решење прегледом изворног кода клијента.

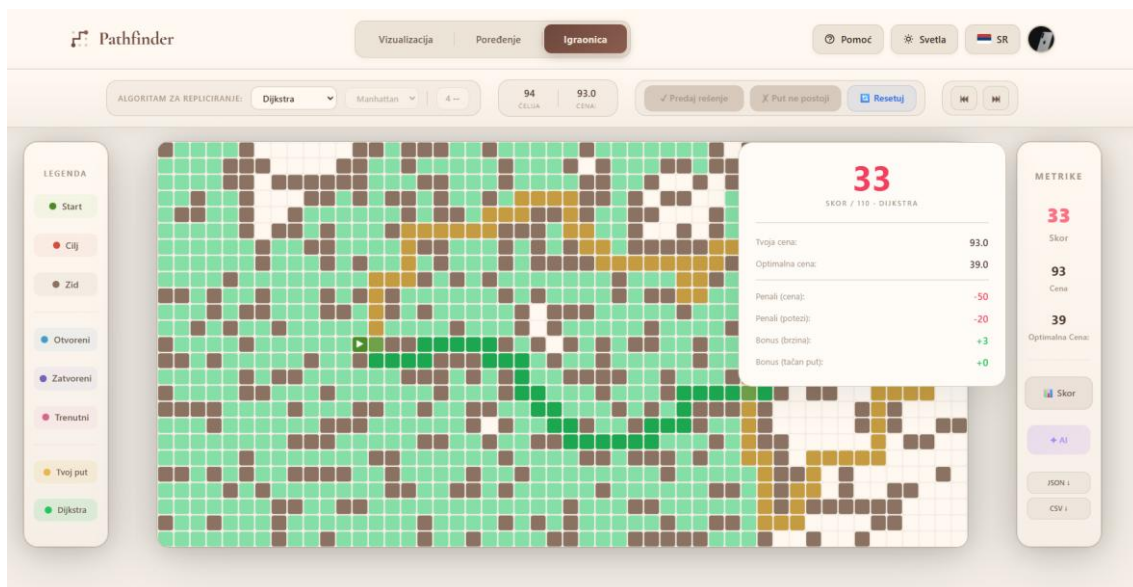
Формула за израчунавање поена дата је изразом који следи.

$$S = \max(0, \min(110, 100 - P_c - P_n + B_b + B_p)) \quad (5.4.1)$$

Полазна вредност износи сто поена. Величина P_c представља казну за одступање од референтне цене и рачуна се као заокружени однос вишка цене и референтне цене помножен са сто, ограничен на педесет поена. Величина P_n представља казну за неисправне потезе, десет поена по потезу, такође ограничену на педесет. Величина B_b представља бонус за брзину у распону од нула до десет поена, који линеарно опада са утрошеним временом. Величина B_p представља бонус од десет поена који се додељује када корисников пут кошта исто или мање од референтног.

Посебно су обрађени гранични случајеви. Уколико пут не постоји, а корисник то тачно закључи, добија сто поена увећано за бонус за брзину, док у случају погрешног закључка у оба смера добија нула поена. Издвојено је и понашање када је референтни алгоритам претрага у дубину. Тај поступак не гарантује оптималност, па се казна за одступање не примењује. Бодује се проналажење било ког исправног пута. Ово представља намерну пројектну одлуку. Не би имало смисла кажњавати корисника због скупљег пута када ни сам референтни поступак не обећава најјефтинији.

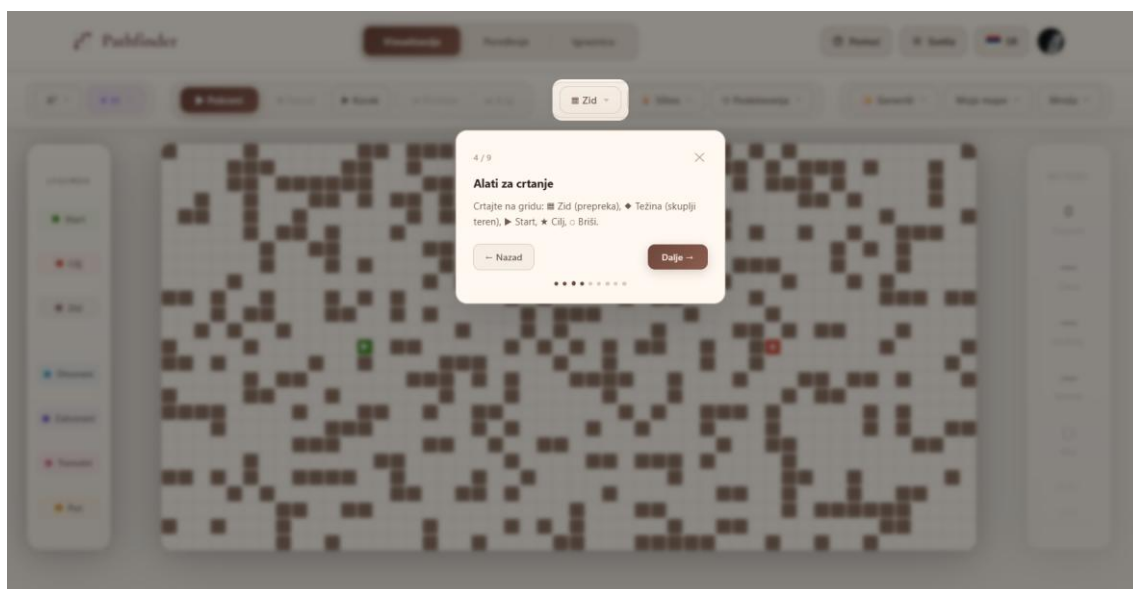
По завршетку бодовања приказ прелази у режим сличан режиму поређења, у коме су истовремено видљиви корисников пут и подручје које је обрадио референтни алгоритам, у различитим бојама. Десна површина приказује укупан број поена, разлагање на појединачне саставнике и поређење цене корисниковог пута са референтном, што је приказано на слици 5.4.2. Разлагање поена уведено је намерно, зато што корисник тако види због чега је изгубио поене, чиме се оцена претвара у повратну информацију. Резултат се потом уписује у базу и објављује свим повезаним корисницима преко утичнице, због чега се табела најбољих резултата ажурира без освежавања странице.



Слика 5.4.2. Оцењивање корисниковог решења у режиму полигона

5.5. Интерактивни водич и кориснички налог

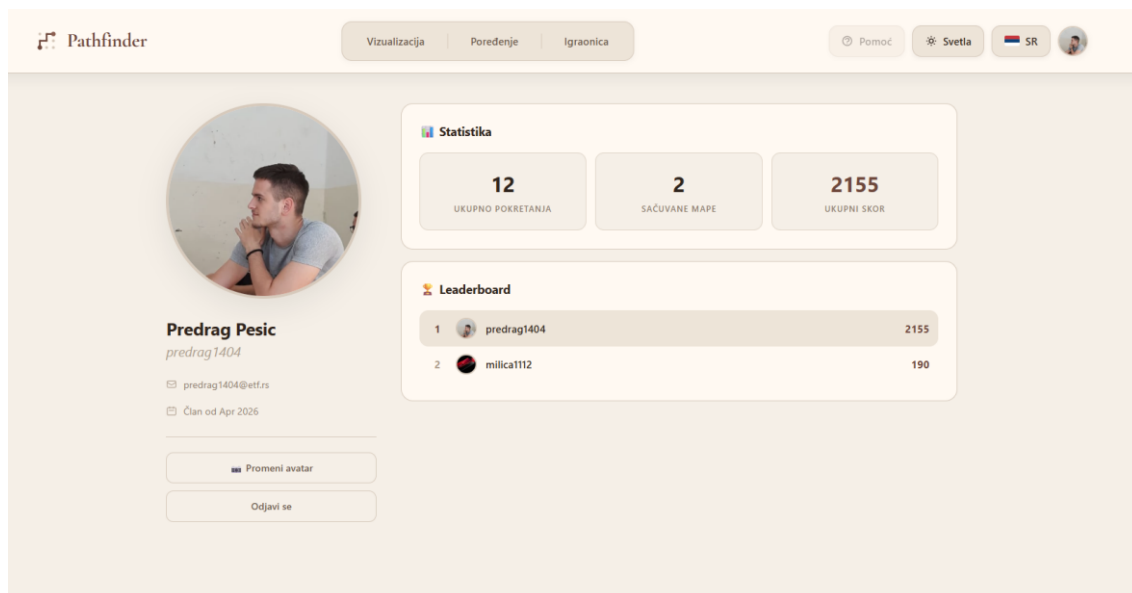
За кориснике који систем виде први пут реализован је водич који кроз интерфејс води корак по корак. Прилагођен је свакој страници и укупно садржи осамнаест корака: девет на страници за визуелизацију, пет на страници за поређење и четири на страници полигона. Водич истиче један елемент интерфејса тако што остатак екрана затамни, уз изузимање области око истакнутог елемента. Пошто се неки елементи приказују тек након што се одређени део интерфејса отвори, уведен је механизам поновних покушаја проналажења елемента. Водич се може затворити у сваком тренутку, без губитка нацртане мапе, а његов изглед приказан је на слици 5.5.1.



Слика 5.5.1. Интерактивни водич кроз апликацију

Регистрација и пријава омогућавају чување мапа и праћење резултата. Страница профила приказује основне податке, слику профила и статистику покушаја у режиму

полигона, што је приказано на слици 5.5.2. Слика профила шаље се на сервис за складиштење слика, а сервер обавља посредовање. Мапе се чувају под називом који корисник задаје и могу бити приватне или јавне, чиме је омогућено да наставник припреми скуп мапа за вежбу.



Слика 5.5.2. Кориснички профил и статистика

Ово поглавље приказало је режиме рада и начин на који су у њима остварени захтеви уочени у поглављу 3. Режим поређења покрива празнину у погледу упоредне анализе, приказ мерених величина празнину у погледу мерења, а режим полигона празнину у погледу активног учешћа корисника. Наредно поглавље описује слој вештачке интелигенције.

6. ИНТЕГРАЦИЈА ВЕШТАЧКЕ ИНТЕЛИГЕНЦИЈЕ

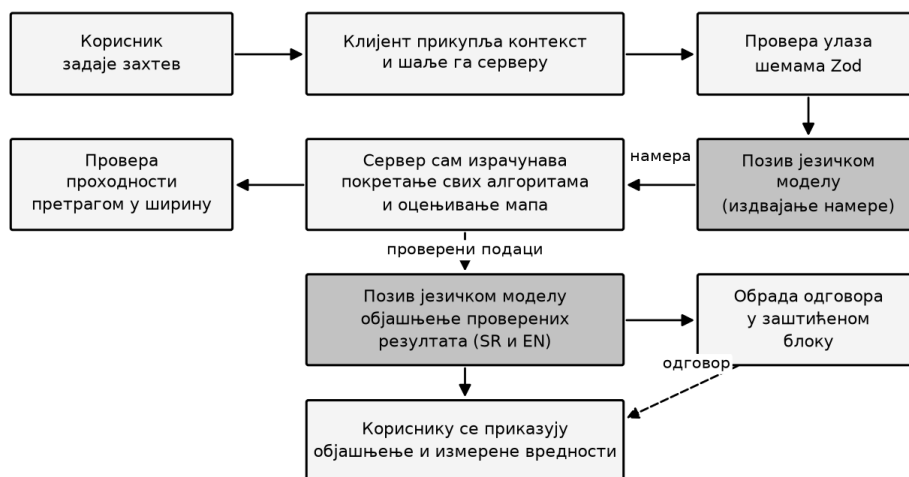
Велики језички модели, засновани на архитектури трансформера [31], последњих година показали су способност да обављају задатке за које нису били посебно обучени [32]. У наставном софтверу њихова употребљивост произлази из способности да сложену појаву објасне природним језиком прилагођеним контексту. Са друге стране, познато је да ови модели показују слабости у нумеричком закључивању [33], што њихову непосредну употребу у мерљивим задацима чини несигурном. Ово поглавље описује како је наведено ограничење обликовало архитектуру слоја вештачке интелигенције, описује четири реализована модула и разматра механизме заштите.

6.1. Приступ и архитектура слоја

Основна пројектна одлука гласи да језички модел никада не даје податак који систем може сам да израчуна. Његова улога сведена је на разумевање захтева израженог природним језиком и на објашњавање већ проверених резултата. Свака бројчана тврдња која се приказује кориснику потиче из стварног покретања алгоритма, а не из одговора модела.

Ова одлука има непосредно упориште у мерењу изложеном у поглављу 9. Испитивање је обухватило задатак у коме модел треба само да из табеле резултата прочита који је алгоритам обрадио најмање чворова. Тачност у зависности од модела износи између 35,7 и 90,0 одсто. Другим речима, задатак који је за програм тривијалан за језички модел није поуздан, па би приказивање таквог одговора као чињенице у наставном контексту повремено значило приказивање неистине.

Сви позиви усмерени су кроз сервер, како је приказано на слици 6.1.1. Клијент упућује захтев сопственом серверу, сервер саставља упит, позива спољашњи сервис и обрађује одговор. Оваква организација штити приступни кључ, омогућава ограничавање броја позива и, што је овде најважније, оставља простор да сервер пре или после позива изврши сопствено израчунавање. Улазни подаци сваког од шест сучеља проверавају се шемама библиотеке Zod пре обраде.



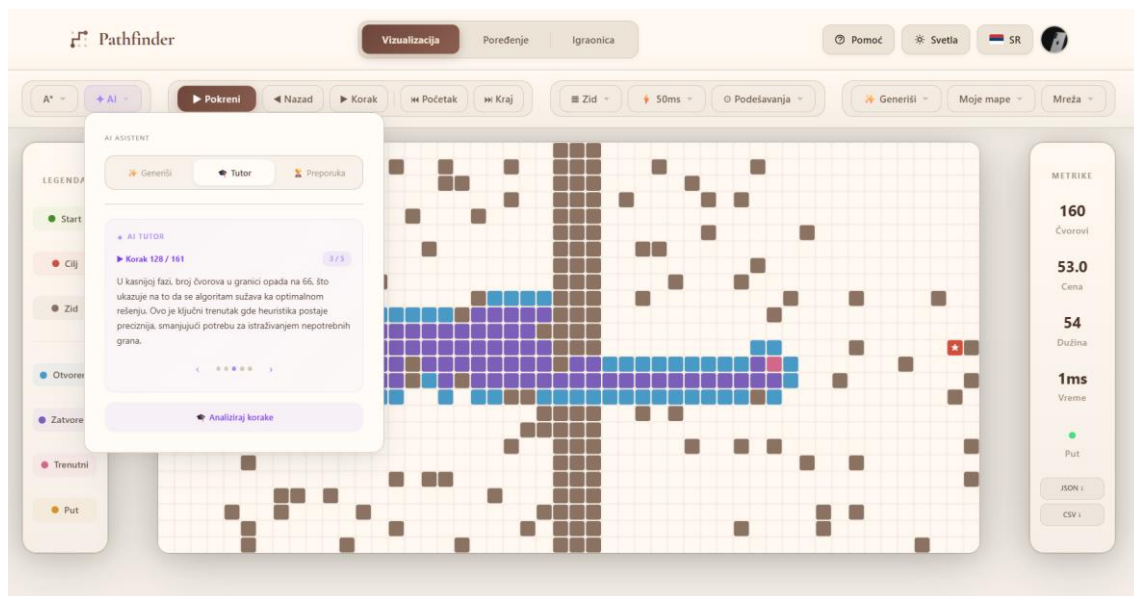
Тамније означени кораци захтевају позив спољашњем сервису.

Слика 6.1.1. Ток обраде захтева у слоју вештачке интелигенције

6.2. Титор

Модул титора објашњава ток претраге након што је она завршена. Наивна замисао била би да се целокупан траг пошаље моделу и да се од њега затражи да изабере занимљиве тренутке, али је такав приступ одбачен из два разлога. Први је обим, пошто траг на мрежи стандардних димензија садржи више хиљада догађаја. Други је поузданост. Избор тренутака зависио би од модела, па исти алгоритам на истој мапи не би увек давао исто објашњење.

Усвојено решење раздваја избор од објашњења. Клијент програмски издваја пет карактеристичних тренутака из трага: почетак претраге, тренутак највећег frontiјера, средину претраге, касну фазу и тренутак проналажења пута. За сваки од њих прикупља и контекст, односно редни број корака, број до тада проширених чворова, тренутну величину frontiјера и назив алгоритма. Модел добија само тај сажетак и његов задатак сведен је на састављање објашњења. Пошто индекси корака потичу из програма, дугме за прелазак на описани корак увек ради исправно. Мерење изложено у одељку 9.4 показало је да модели у 99,1 одсто случајева врате исправно обликован одговор који чува задате индексе, што овај модул чини најпоузданијим од свих. Изглед панела приказан је на слици 6.2.1.



Слика 6.2.1. Панел татора са објашњењима кључних тренутака

6.3. Генератор сценарија

Модул генератора омогућава да корисник опише жељени сценарио природним језиком, на пример захтевом за мапом на којој је алгоритам A^* знатно бржи од Дајкстриног алгоритма. Реализација овог модула прошла је кроз битну промену током развоја, која добро илуструје образложену пројектну одлуку.

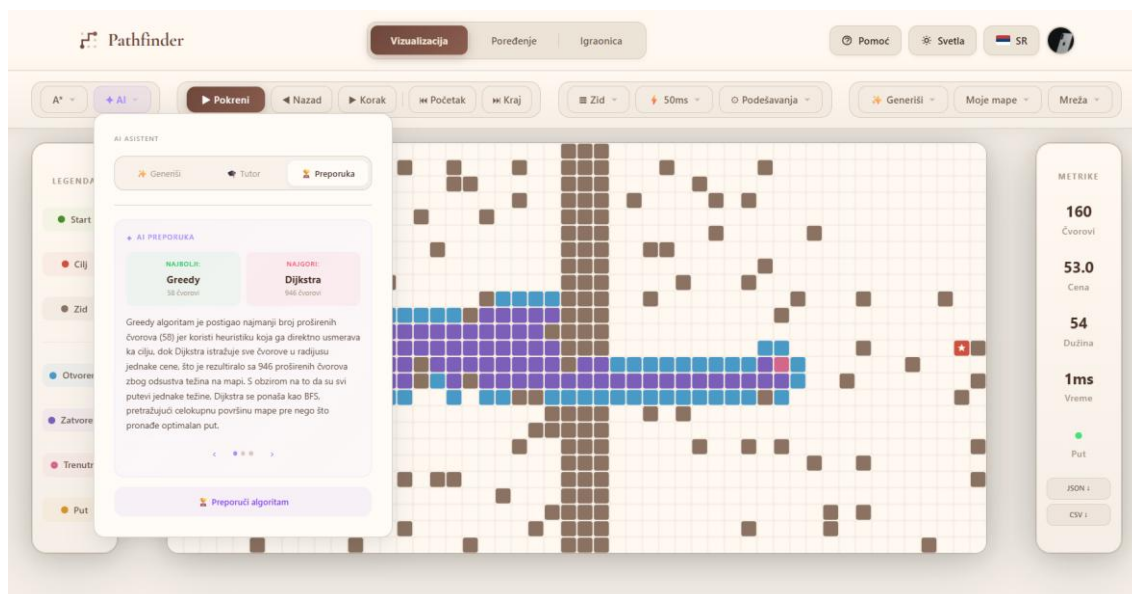
Првобитна замисао била је да модел непосредно произведе саму мрежу, са списком зидова и тежина, што се показало неупотребљивим. Модел је враћао мреже које јесу биле синтаксно исправне, али које најчешће нису задовољавале постављени захтев, а нередовно се дешавало и да пут између полазног и циљног чвора уопште не постоји. Разлог је јасан, јер модел не може да предвиди како ће се алгоритми понашати на мрежи коју управо ствара.

Усвојено решење има три степена. У првом степену модел из корисниковог захтева издваја структуриран опис намере: врсту захтева и списак алгоритама на које се он односи. Подржано је пет врста намере. У другом степену сервер генерише више од двадесет кандидат мапа, комбинујући седам генератора са више вредности зрна случајног низа. Затим покреће свих осам алгоритама на свакој мапи и оцењује је бројчано према издвојеној намери. Проходност се додатно проверава покретањем претраге у ширину. У трећем степену изабрана мапа, заједно са стварно измереним вредностима, прослеђује се моделу који саставља објашњење, и то напореда на српском и енглеском језику. Кориснику се приказују и објашњење и табела измерених вредности, чиме тврдња да је један алгоритам бржи од другог не остаје на нивоу изјаве модела.

6.4. Препоручивач и контекстуална помоћ

Модул препоручивача одговара на питање који је алгоритам најпогоднији за мапу коју корисник тренутно има пред собом. Ток обраде поново прати исти образац. Клијент шаље опис мапе, а сервер покреће свих осам алгоритама. Затим ствара измењену варијанту мапе, тако што уклони четвртину зидова или дода пондерисан терен, и покреће алгоритме и на њој. Тек након тога модел добија обе табеле резултата и саставља објашњење. Кориснику се приказују три целине: најбољи и најлошији алгоритам издвојени на основу измерених

вредности, савет о томе шта на мапи узрокује такав исход, и приказ утицаја који би имала измена мапе. Изглед панела приказан је на слици 6.4.1.



Слика 6.4.1. Панел препоручивача алгорита

Четврти модул нуди кратка објашњења на захтев, тако што клик на приказану вредност, на пример на број проширених чворова, приказује објашњење у облику облачића. Обим одговора намерно је ограничен и није предвиђен слободан разговор, зато што би он одвлачио пажњу са предмета учења и отварао простор за одговоре које систем не може да провери. У истом модулу реализован је и сажети коментар након режима поређења.

6.5. Робусност и подела одговорности

Ослањање на спољашњи сервис уводи начине отказивања који не постоје у остатку система, па су предузете четири мере. Свака обрада одговора обавијена је провером. Неисправан одговор доводи до контролисане поруке о грешци, а не до прекида рада. Ова мера показала се оправданом када је погрешно наведена ознака модела довела до тога да ниједан од двадесет захтева не буде успешно обрађен. Позивима је постављено временско ограничење од шездесет секунди. Одговори се траже у машински читљивом облику уз опис очекиване структуре, чиме је удео исправно обликованих одговора достигао 99,8 одсто. Двојезични одговори добијају се напоредним позивима, па објашњење на српском језику није дослован превод.

Табела 6.5.1 сажима поделу одговорности између програма и језичког модела у сваком модулу. Из ње се уочава да ниједан модул не препушта моделу израчунавање, а ова доследно спроведена подела представља главни допринос овог дела рада.

Табела 6.5.1. Подела одговорности у модулима вештачке интелигенције

Модул	Задатак програма	Задатак језичког модела
тутор	издвајање карактеристичних тренутака из трага	састављање објашњења за задате тренутке
генератор	стварање и оцењивање кандидат мапа	издвајање намере, објашњење исхода

Модул	Задатак програма	Задатак језичког модела
препоручивач	покретање свих алгоритама и варијанте мапе	тумачење измерених вредности
контекстуална помоћ	прикупљање контекста са екрана	кратко објашњење појма

Ово поглавље описало је слој вештачке интелигенције и образложило приступ у коме модел објашњава, а програм рачуна. Наредно поглавље прелази на методологију мерења.

7. МЕТОДОЛОГИЈА ЕВАЛУАЦИЈЕ

Да би резултати изложени у наредним поглављима били проверљиви, потребно је прецизно описати шта је мерено, како је мерено и под којим условима. Ово поглавље дефинише мерене величине, описује инфраструктуру изграђену за спровођење мерења, наводи услове мерења и објашњава начин статистичке обраде, а завршава се разматрањем ограничења примењене методологије.

7.1. Мерене величине

Приликом одабира мерених величина пошло се од запажања да време извршавања, иако најочигледнија мера, зависи од рачунара, оптерећења система и начина реализације, због чега само по себи не даје упоредиву слику. Због тога је уведено пет величина, наведених у табели 7.1.1.

Табела 7.1.1. Мерене величине и њихово значење

Величина	Ознака	Јединица	Значење
број проширених чворова	E	чвор	колико је чворова алгоритам обрадио до заустављања
максимална величина фронтијера	M	чвор	највећи истовремени број чворова који чекају обраду
цена пронађеног пута	C	бездимензионална	збир тежина ћелија дуж пронађеног пута
дужина пронађеног пута	L	корак	број ћелија на путу
време извршавања	t	ms	протекло време од почетка до заустављања претраге

Број проширених чворова изабран је за основну меру ефикасности, јер не зависи ни од брзине рачунара ни од квалитета реализације. Ова претпоставка накнадно је проверена, што је изложено у одељку 8.7. Максимална величина фронтијера уведена је као посредна мера потрошње меморије и бележи највећу вредност коју у току претраге достигне број чворова у реду, стеку или приоритетном реду.

Приликом припреме мерења уочен је проблем који је захтевао посебно решење. Алгоритми интерно рачунају различите величине: претрага у ширину води евиденцију о броју корака, Дајкстрин алгоритам о збиру тежина, а претрага 0-1 у ширину о броју грана са ценом један. Уколико би се као цена пута забележила интерна вредност сваког алгоритма, добијени бројеви међусобно не би били упоредиви. Најизраженији пример представља претрага 0-1 у ширину, која на мапама са јединичним тежинама пријављује цену нула. Решење је да се цена не преузима од алгоритма, већ да се измери накнадно на путу који је алгоритам вратио. Пут се реконструира из мапе родитеља, а затим се саберу тежине терена свих ћелија кроз које пролази, уз множење дијагоналних корака чиниоцем $\sqrt{2}$. Тако добијена

величина, у наставку названа стварна цена пута, дефинисана је на исти начин за све алгоритме.

Апсолутна цена пута зависи од мапе, па је као мера квалитета уведена релативна величина. За сваку мапу израчунава се референтна цена C_{opt} покретањем Дајкстриног алгоритма, за који је доказано да враћа најјефтинији пут, након чега се субоптималност рачуна изразом који следи.

$$\sigma = (C - C_{opt}) / C_{opt} \cdot 100 \% \quad (7.1.1)$$

Вредност нула значи да је алгоритам пронашао најјефтинији пут, а вредност од сто одсто да је пронађени пут два пута скупљи. Ова величина омогућава поређење на мапама различитих димензија и различите густине препрека.

7.2. Инфраструктура и категорије мерења

За спровођење мерења изграђен је засебан подсистем, независан од корисничког интерфејса. Њега чине: генератори мапа описани у одељку 4.5, посебне реализације алгоритама прилагођене мерењу, дефиниције сценарија, скрипт за покретање из командне линије и модул за извоз резултата. Реализације у овом подсистему не бележе траг извршавања, јер би бележење више хиљада догађаја по симулацији вишеструко продужило мерење. Приоритетни ред реализован је као бинарни хип, на исти начин као у клијентској реализацији. Измерена времена тако одражавају својства алгоритама, а не својства структуре података. Резултати се уписују у базу и истовремено извозе у формате CSV и JSON, а сваки покренути скуп добија сопствену ознаку. Наредбе за покретање наведене су у прилогу А.

Дефинисано је шест категорија, наведених у табели 7.2.1. Свака од њих мења једну групу параметара, а остале држи непроменљивим, чиме се утицај појединачног чиниоца може издвојити.

Табела 7.2.1. Категорије спроведених мерења

Ознака	Предмет испитивања	Број симулација
E1	понашање на седам типова мапа при три густине препрека	2640
E6	скалабилност при пет величина мреже	2160
E7	утицај избора хеуристичке функције	1800
E8	поређење суседства са четири и осам суседа	1440
E9	утицај тежинског параметра w	972
E10	понашање на мапама без решења	432
укупно		9444

Категорија E10 заслужује напомену, пошто су мапе без решења добијене тако што су све ћелије суседне полазном чвору означене као зид, па пут по конструкцији не постоји. Ова категорија не мери ефикасност, већ проверава исправност, то јест да ли се сваки алгоритам заустави и исправно пријави да пут не постоји.

7.3. Услови мерења и обрада резултата

Сва мерења обављена су на једном рачунару са процесором AMD Ryzen 9 7900 и 64 GB радне меморије. Оперативни систем био је Windows 11, а извршно окружење Node.js

24.12. Мерења су текла у једном процесу и једној нити, а на систему није било других активних оптерећења. Мерење времена обавља се уграђеним бројачем високе резолуције.

Пре почетка сваке категорије извршава се фаза загревања, у оквиру које се сваки алгоритам покрене тридесет пута над помоћном мапом, а добијени резултати се одбацују. Ова мера уведена је након што је уочено да прва мерења дају вишеструко веће вредности од каснијих, што је последица накнадног превођења кода при извршавању. Након увођења загревања, највећа забележена вредност за Дајкстрин алгоритам на мрежи стандардних димензија смањила се са 5,46 на 0,49 милисекунди, чиме је артефакт хладног старта уклоњен. Поновљивост је обезбеђена тиме што сваки генератор прима зрно случајног низа. Генератор псеудослучајних бројева реализован је као линеарни конгруентни генератор са непроменљивим параметрима. Због тога се било које наведено мерење може поновити под истим условима.

Обрада измерених података обављена је засебним скриптом написаним у језику Python. Скрипт учитава извезену датотеку, израчунава све изведене величине, саставља табеле и генерише графике. Тако је обезбеђено да бројеви наведени у тексту потичу непосредно из обраде. За сваку аритметичку средину израчунате су стандардна девијација и полуширина 95 одсто интервала поверења, према изразу који следи. У том изразу s је узорачка стандардна девијација, n број мерења, а t вредност Студентове расподеле са $n - 1$ степени слободе.

$$\Delta = t_{0,975; n-1} \cdot s / \sqrt{n} \quad (7.3.1)$$

Времена извршавања приказују се медијаном, а не аритметичком средином, због асиметричне расподеле ове величине. Графички прикази обликовани су тако да остану читљиви и у црно-белој штампи. Линије се разликују обликом ознаке и типом линије, а површине у стубичастим дијаграмима шрафуром.

7.4. Ограничења методологије

Приликом тумачења резултата треба имати у виду три ограничења. Прво, мапе су генерисане поступцима описаним у одељку 4.5 и представљају одређену класу проблема, па се резултати не морају пренети на мапе битно другачијих својстава. У литератури постоји стандардизована збирка мапа [16], чија би употреба омогућила поређење са објављеним резултатима. Она ипак није коришћена, пошто њене мапе не садрже пондерисан терен, који је у овом раду суштински. Друго, време извршавања измерено је на једном рачунару и у једном извршном окружењу, па апсолутне вредности не треба преносити на друге системе, док односи међу алгоритмима остају информативни. Треће, реализације су писане тако да буду читљиве и међусобно упоредиве, а не да буду што бржи могући. Оптимизована реализација дала би краће време, а број проширених чворова остао би непромењен.

Ово поглавље описало је шта је мерено и под којим условима. Уведене су заједничка скала цене пута и мера субоптималности. Наредно поглавље излаже добијене резултате.

8. РЕЗУЛТАТИ ЕВАЛУАЦИЈЕ АЛГОРИТАМА

Ово поглавље излаже резултате добијене на 9444 симулације спроведене према методологији описаној у претходном поглављу. Излагање прати редослед категорија мерења. Најпре се даје збирни преглед, затим понашање по типовима мапа, скалабилност, утицај хеуристике, суседства и тежинског параметра. На крају следе меморијски отисак и расправа.

8.1. Збирни преглед

Табела 8.1.1 приказује збирне резултате за седам типова мапа при три густине препрека, а свака вредност добијена је из 330 симулација по алгоритму.

Табела 8.1.1. Збирни преглед свих алгоритама на мрежама димензија 25×50

Алгоритам	N	Проширења	95% ИП ($\pm$)	Цена пута	Субопт. (%)	Макс. фронтјер	Време (ms)
BFS	330	583,11	29,47	74,58	18,11	29,33	0,18
DFS	330	276,45	27,63	235,09	378,15	145,49	0,06
Dijkstra	330	584,11	29,98	66,48	0,00	41,79	0,26
A*	330	218,74	17,82	66,48	0,00	51,38	0,07
Greedy	330	95,11	11,80	77,89	23,89	37,68	0,02
Swarm	330	123,50	13,79	67,76	2,97	39,50	0,03
Conv. Swarm	330	101,07	12,42	73,16	15,51	37,15	0,02
0-1 BFS	330	310,54	26,32	122,03	128,51	115,38	0,09

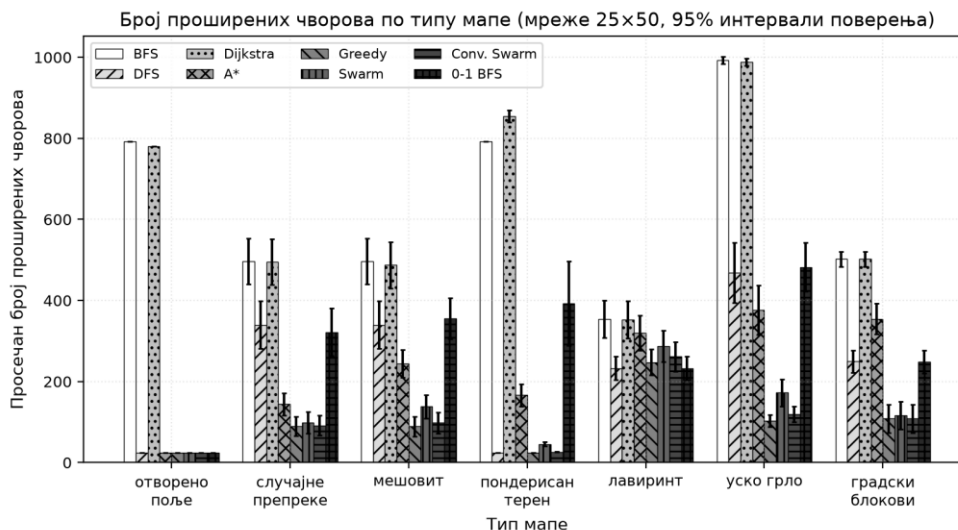
Уочавају се четири целине. Дајкстрин алгоритам и алгоритам A* дају нулту субоптималност, што представља експерименталну потврду теоријске гаранције из одељака 2.3 и 2.4. Значајно је да алгоритам A* тај исти резултат постиже уз 218,74 проширена чвора наспрам 584,11 колико их обради Дајкстрин алгоритам. Реч је о 62,5 одсто мање обраде, што представља најјаснију меру вредности коју уноси хеуристика.

Другу целину чине пондерисане варијанте. Варијанта Swarm обради 123,50 чворова, 43,5 одсто мање него алгоритам A*, а цена коју за то плаћа износи свега 2,97 одсто субоптималности. Варијанта Convergent Swarm уз 101,07 проширења достиже 15,51 одсто субоптималности. Поређење те две варијанте показује да однос између уштеде и губитка није линеаран. Трећу целину чини похлепна претрага, која са 95,11 проширења представља најефикаснији поступак по том мерилу, али уз 23,89 одсто субоптималности. Четврту целину чине поступци чија је субоптималност изразито висока: претрага 0-1 у ширину са 128,51 одсто и претрага у дубину са 378,15 одсто.

Претрага у ширину заслужује посебан коментар. Њена субоптималност од 18,11 одсто на први поглед изгледа као противуречност са тврдњом да овај поступак гарантује најкраћи пут. Противуречности нема, јер она минимизује број корака, а не збир тежина, па на мапама са пондерисаним тереном најкраћи пут по броју корака води кроз скупе ћелије.

8.2. Понашање по типовима мапа

Слика 8.2.1 приказује број проширених чворова по типовима мапа, а табела 8.2.1 субоптималност свих алгоритама по истим типовима.



Слика 8.2.1. Број проширених чворова по типу мапе

Табела 8.2.1. Субоптималност алгоритама по типу мапе, у процентима

Тип мапе	Пут (%)	BFS	DFS	Dijkstra	A*	Greedy	Swarm	Conv. Swarm	0-1 BFS
отворено поље	100,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00
случајне препреке	70,00	0,00	494,38	0,00	0,00	7,65	2,45	6,40	286,80
мешовит	70,00	45,90	906,56	0,00	0,00	58,87	4,43	28,75	33,44
пондерисан терен	100,00	68,99	68,99	0,00	0,00	68,99	7,20	58,67	182,01
лабиринт	100,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00
уско грло	100,00	0,00	258,64	0,00	0,00	7,97	4,89	7,68	135,12
градски блокови	93,33	0,00	197,28	0,00	0,00	1,61	0,60	1,61	197,28

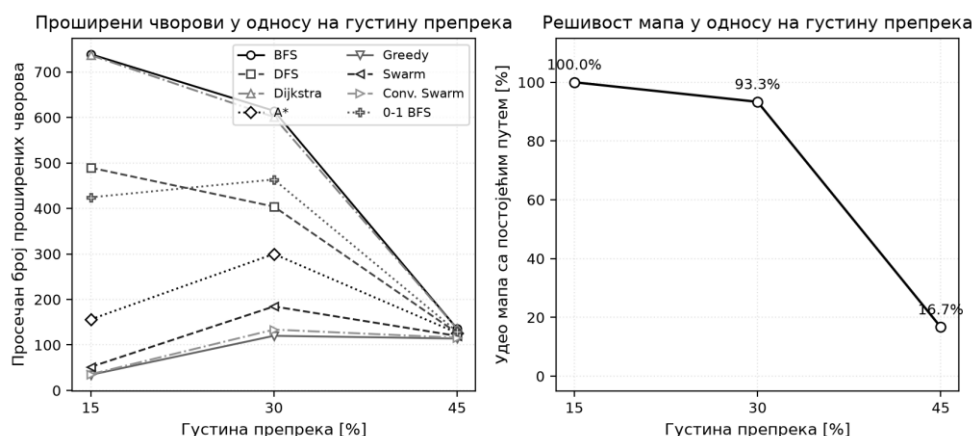
Разлике међу типовима су велике. На отвореном пољу претрага у ширину обради 793 чвора, а алгоритам A* свега 25, тачно онолико колико пут има ћелија. Разлог је у томе што хеуристика на празном простору тачно погађа смер. На мапама типа лавиринт распон је најужи, од 233 до 354 проширења, зато што хеуристика у уском коридору нема простора да усмери претрагу. На мапама са уским пролазом распон је најшири, од 103 до 993.

Три реда табеле посебно се издвајају. На мапама типа лавиринт сви поступци дају нулту субоптималност. То није својство алгоритама већ својство проблема. У лавиринту између полазног и циљног чвора постоји тачно један пут, па сваки поступак који пут пронађе мора пронаћи баш тај. Овај налаз има наставну вредност. Он показује да разлике међу алгоритмима произлазе из односа између алгоритма и структуре проблема.

Ред који одговара пондерисаном терену најоштрије раздваја поступке. Претрага у ширину, претрага у дубину и похлепна претрага дају идентичну субоптималност од 68,99 одсто. Прве две до тога долазе зато што уопште не разликују тежине терена, а похлепна претрага зато што узима у обзир само процену преостале удаљености. Варијанта Swarm постиже врло добар компромис са свега 7,20 одсто. Претрага 0-1 у ширину достиже 182,01

одсто, зато што све тежине мање или једнаке јединици тумачи као нулу и тако ради изван свог модела. Мапа са уским пролазом осмишљена је да провери понашање хеуристике када она усмерава претрагу у привидно исправан, али блокиран смер. Резултат је супротан очекивању: похлепна претрага обради само 103 чвора уз 7,97 одсто субоптималности. Објашњење је у томе што генератор поставља пролаз у средини преграде, управо у смеру у коме хеуристика ионако усмерава претрагу.

Слика 8.2.2 приказује утицај густине препрека и садржи налаз важан за тумачење свих осталих резултата.

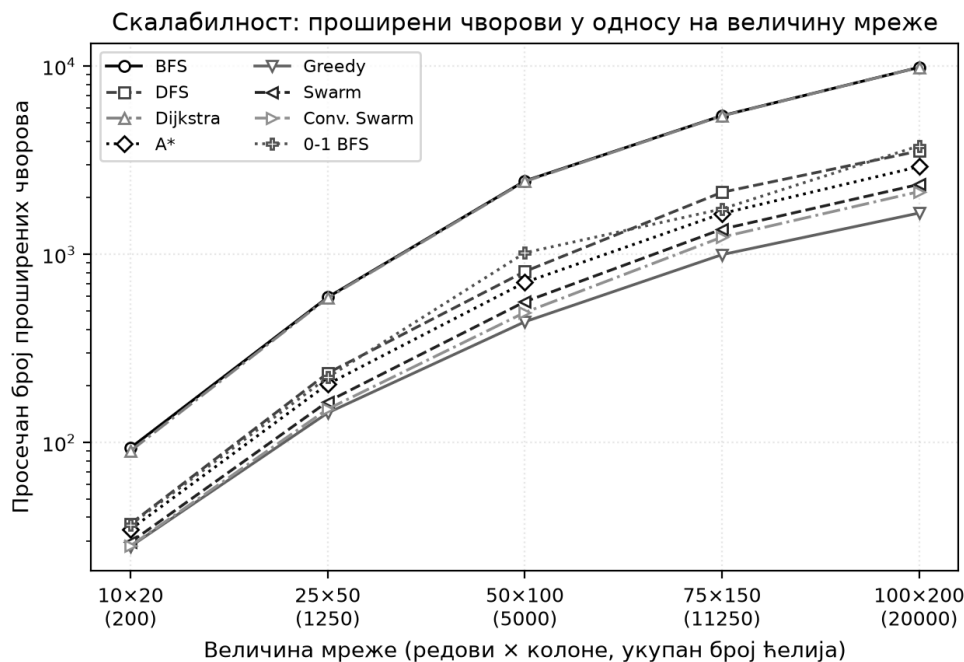


Слика 8.2.2. Утицај густине препрека на број проширења и на решивост мапа

При густини од петнаест одсто пут постоји на свим мапама, при тридесет одсто на 93,33 одсто мапа, а при четрдесет пет одсто на свега 16,67 одсто мапа. Ово нагло опадање одговара појави прага перколације, због чега су просечне вредности за највећу густину израчунате на малом броју решивих мапа и имају мању поузданост. Друга последица велике густине јесте изједначавање алгоритама. При петнаест одсто густине распон броја проширења износи од 34 до 739, а при четрдесет пет одсто свега од 114 до 136, зато што густе препреке саме по себи ограничавају простор претраге.

8.3. Скалабилност

Категорија Е6 испитује понашање при повећању мреже од 200 до 20000 ћелија. Слика 8.3.1 приказује добијене вредности у логаритамским координатама, где степена зависност постаје права линија. Нагиб те линије представља емпиријски експонент раста добијен прилагођавањем модела $E \approx C \cdot N^k$, где је N број ћелија.



Слика 8.3.1. Број проширених чворова у односу на величину мреже

Табела 8.3.1. Емпиријски експонент раста броја проширених чворова

Алгоритам	Експонент раста k	Проширења 10×20	Проширења 100×200	Фактор пораста
BFS	1,011	93,741	9874,685	105,340
DFS	0,991	37,000	3545,167	95,815
Dijkstra	1,019	90,278	9865,444	109,279
A*	0,959	34,593	2929,778	84,694
Greedy	0,882	28,056	1659,204	59,140
Swarm	0,948	29,519	2357,093	79,851
Conv. Swarm	0,937	28,278	2156,519	76,262
0-1 BFS	0,995	36,574	3776,648	103,260

Вредности из табеле 8.3.1 носе јасну поруку. Претрага у ширину и Дајкстрин алгоритам имају експонент близак јединици, што значи да обрађују сталан удео мреже. Похлепна претрага има експонент 0,882, па њен удео обрађене мреже опада како мрежа расте. Алгоритам А* налази се између, са експонентом 0,959. Практична последица види се у чиниоцу пораста. При стоструком повећању мреже Дајкстрин алгоритам повећа обраду 109,3 пута, а похлепна претрага само 59,1 пута. Предност усмерене претраге тако расте са величином проблема. Мерење времена потврђује исти закључак са израженијим разликама, пошто на највећој мрежи Дајкстрин алгоритам захтева 5,67 милисекунди, а похлепна претрага 0,11 милисекунди.

8.4. Утицај хеуристичке функције и типа суседства

Категорија Е7 пореди четири хеуристике на мрежама са четири суседа, а добијене вредности дате су у табели 8.4.1.

Табела 8.4.1. Број проширења по изабраној хеуристичкој функцији

Алгоритам	Манхетн	еуклидска	Чебишевљева	октилна	Субопт. (%)
-----------	---------	-----------	-------------	---------	-------------

Алгоритам	Манхетн	еуклидска	Чебишевљева	октилна	Субопт. (%)
A*	224,19	295,33	320,42	273,63	0,00
Greedy	110,36	101,79	104,51	102,63	17,26
Swarm	139,00	129,88	133,47	130,94	1,80
Conv. Swarm	117,56	109,96	110,20	111,13	9,51

Резултати за алгоритам A* потпуно одговарају теорији. Манхетн хеуристика даје најмањи број проширења, 224,19. У режиму са четири суседа она даје тачно растојање, па је од свих допустивих најинформативнија. Октилна даје 273,63, еуклидска 295,33, а Чебишевљева 320,42 проширења. Све четири дају идентичну цену пута, то јест нулту субоптималност, чиме је потврђено да допустивост чува гаранцију оптималности независно од тога колико је процена груба.

Код похлепне претраге редослед се обрће. Најмањи број проширења даје еуклидска хеуристика са 101,79, а највећи Манхетн са 110,36. Објашњење је у томе што похлепна претрага не поседује податак о пређеном путу, па целокупан њен избор зависи од процене. Еуклидска хеуристика мења се глатко у простору и међу суседним чворовима ретко даје једнаке вредности, док Манхетн хеуристика на мрежи производи читаве области са истом вредношћу, унутар којих претрага нема основ за избор. Општи закључак јесте да оштрија хеуристика користи алгоритму A*, а глаткија похлепној претрази.

Категорија E8 пореди режиме са четири и са осам суседа, а добијене вредности дате су у табели 8.4.2.

Табела 8.4.2. Поређење суседства са четири и са осам суседа

Алгоритам	Проширења 4	Проширења 8	Цена 4	Цена 8	Дужина 4	Дужина 8
BFS	711,20	782,54	82,55	69,56	76,00	55,48
DFS	245,94	408,60	130,32	828,20	123,77	392,96
Dijkstra	719,79	753,90	77,22	62,59	77,41	55,51
A*	224,19	149,70	77,22	62,65	77,41	55,54
Greedy	110,36	73,66	84,18	70,41	77,64	56,04
Swarm	139,00	90,66	78,27	64,00	77,89	55,66
Conv. Swarm	117,56	78,47	83,25	66,40	77,57	55,68
0-1 BFS	314,26	314,73	120,40	120,41	121,16	121,29

Три налаза издвајају се као значајна. Први је да режим са осам суседа даје знатно краће путеве, 55,51 корак наспрам 77,41. То одговара смањењу од 28,3 одсто, зато што дијагонални корак покрива померање по обе осе истовремено. Други се односи на решивост. У режиму са четири суседа пут постоји на 97,78 одсто мапа, а у режиму са осам суседа на 100 одсто. Тиме је потврђена претпоставка из одељка 2.1 према којој дијагонални прелаз може проћи између два зида који се додирују само у теменима.

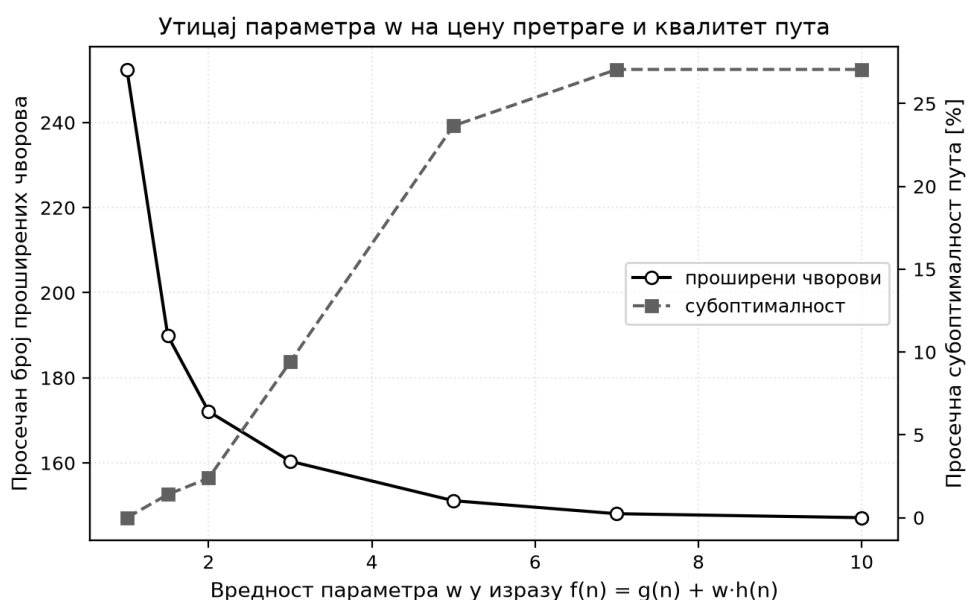
Трећи налаз односи се на алгоритам A*, који у режиму са осам суседа обради 149,70 чворова, мање него у режиму са четири суседа где обради 224,19. Овај налаз захтева опрез. У оба режима коришћена је Манхетн хеуристика, која у режиму са осам суседа прецењује стварно растојање и тиме престаје да буде допустива. Смањење броја проширења зато делом потиче од дијагоналних пречица, а делом од тога што недопустива хеуристика делује као пондерисана варијанта. У прилог томе говори и податак да цена пута у том режиму износи 62,65 наспрам 62,59 колико даје Дајкстрин алгоритам.

8.5. Утицај тежинског параметра

Категорија Е9 испитује утицај параметра w у изразу $f(n) = g(n) + w \cdot h(n)$, а добијене вредности дате су у табели 8.5.1 и на слици 8.5.1.

Табела 8.5.1. Утицај параметра w на број проширења и на квалитет пута

w	Проширења	Цена пута	Субоптималност (%)
1,00	252,48	101,94	0,00
1,50	189,91	102,46	1,40
2,00	172,09	102,81	2,39
3,00	160,39	105,42	9,40
5,00	151,11	110,58	23,66
7,00	148,06	111,92	27,06
10,00	147,13	111,92	27,06



Слика 8.5.1. Утицај параметра w на цену претраге и квалитет пута

Резултати показују три јасне области. За $w = 1$ понашање је идентично алгоритму A^* , уз 252,48 проширења и нулту субоптималност. При $w = 2$ број проширења пада на 172,09, за 31,8 одсто, а субоптималност расте на свега 2,39 одсто. У области између $w = 1,5$ и $w = 3$ добија се значајна уштеда уз малу цену. Изнад $w = 5$ однос се погоршава. Између $w = 7$ и $w = 10$ јавља се засићење: број проширења пада за мање од једног одсто, док субоптималност остаје на око 27 одсто. Разлог је у томе што хеуристика потпуно надјача допринос пређеног пута. Ова мерења пружају и практичну потврду теоријске границе из одељка 2.5. За $w = 2$ та граница износи сто одсто субоптималности, а измерена вредност је 2,39 одсто. Тиме је показано да је граница веома конзервативна.

8.6. Меморијски отисак и провера основне мере

Табела 8.6.1 приказује измерене вредности максималне величине frontiјера.

Табела 8.6.1. Максимална величина frontiјера по алгоритму

Алгоритам	Просек	95% ИП ($\pm$)	Максимум	Однос према броју проширења
BFS	29,33	1,88	51	0,05
DFS	145,49	17,14	702	0,53
Dijkstra	41,79	3,57	136	0,07
A*	51,38	3,62	137	0,23
Greedy	37,68	2,52	109	0,40
Swarm	39,50	2,61	101	0,32
Conv. Swarm	37,15	2,45	100	0,37
0-1 BFS	115,38	11,99	466	0,37

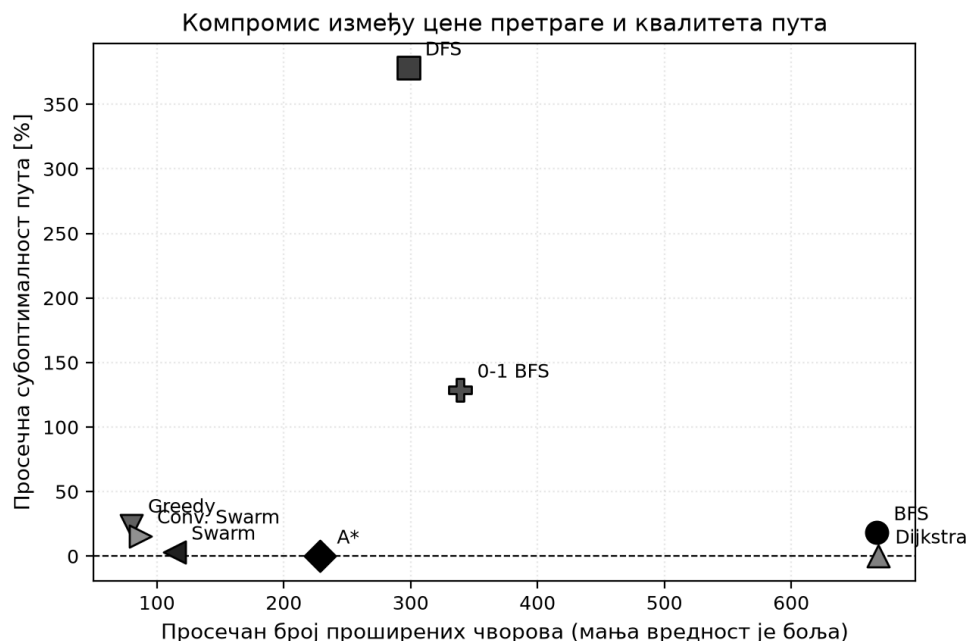
Ова мерења дала су резултат који одступа од уобичајеног очекивања. Претрага у ширину има најмањи меморијски отисак од свих поступака, 29,33 чвора. Претрага у дубину има највећи, 145,49 чворова са забележеним максимумом од 702. Представа изведена из анализе стабла претраге управо је супротна.

Објашњење лежи у разлици између стабла и графа. У стаблу претрага у дубину чува само чворове на текућем путу. У графу са циклусима реализација на стек ставља све још непосећене суседе сваког чвора на путу. Стек зато расте сразмерно пређеном путу помноженом разгранатости. Претрага у ширину, насупрот томе, чува само једну контуру чворова на текућем растојању. Мерење скалабилности потврђује ову појаву. На највећој мрежи стек претраге у дубину достиже 824,52 уноса, док ред претраге у ширину достиже 121,72. Претрага 0-1 у ширину такође показује велики отисак од 115,38 чворова, али из другог разлога, јер њена структура допушта да исти чвор буде уписан више пута.

Провера претпоставке из одељка 7.1, према којој број проширених чворова не зависи од рачунара, спроведена је на узорку од 4800 симулација. Спирманов коефицијент корелације ранга износи $\rho = 0,974$ уз $p < 0,001$, а Пирсонов коефицијент $r = 0,930$. Тиме је потврђено да поредак алгоритама по броју проширења готово потпуно одговара поретку по времену извршавања. Закључци се стога могу пренети на друге рачунаре. Категорија E10 обухвата 432 симулације на мапама без решења. Она је потврдила да сви алгоритми исправно утврђују да пут не постоји и да проширују тачно један чвор.

8.7. Расправа

Слика 8.7.1 сажима централни налаз овог поглавља, приказујући сваки алгоритам као тачку у равни коју чине цена претраге и квалитет пута. Из приказа се уочава да не постоји поступак који је најбољи по оба мерила. Дајкстрин алгоритам и алгоритам A* налазе се на линији нулте субоптималности, а алгоритам A* доминира тиме што исти квалитет постиже уз мању цену. Породица пондерисаних варијанти и похлепна претрага размештени су дуж криве на којој се уштеда плаћа квалитетом. Претрага у дубину и претрага 0-1 у ширину налазе се изван те криве.



Слика 8.7.1. Компромис између цене претраге и квалитета пута

На основу изложених резултата могу се формулисати следеће препоруке. Када је потребан доказано најјефтинији пут, алгоритам A* са допустивом хеуристиком представља најбољи избор, јер даје исти резултат као Дајкстрин алгоритам уз 62,5 одсто мање обраде. Када се мало одступање може прихватити, варијанта Swarm нуди најповољнији однос, додатну уштеду од 43,5 одсто уз мање од три одсто субоптималности, док Дајкстрин алгоритам остаје неопходан када хеуристика није доступна.

Три налаза заслужују посебно истицање, јер их уобичајено излагање материје не садржи. Први је да претрага у ширину показује мерљиву субоптималност од 18,11 одсто, што њену гаранцију ставља у прави контекст. Други је да претрага у дубину троши више меморије од претраге у ширину када се примени на граф, што је супротно закључку изведеном за стабло. Трећи је да претрага 0-1 у ширину, примењена изван скупа тежина за који је намењена, даје субоптималност од 128,51 одсто иако је формално исправно реализована. Сва три налаза поткрепљују општу тврдњу да својства алгоритма важе само унутар модела за који су изведена. Наредно поглавље прелази на вредновање компоненте вештачке интелигенције и корисничког интерфејса.

9. ЕВАЛУАЦИЈА ВЕШТАЧКЕ ИНТЕЛИГЕНЦИЈЕ И КОРИСНИЧКОГ ИНТЕРФЕЈСА

Ово поглавље излаже резултате вредновања двају преосталих делова система. Први део посвећен је слоју заснованом на језичким моделима и уводи меру која исправља методолошки недостатак уочен током обраде. Други део посвећен је едукативној компоненти и корисничком интерфејсу: верификацији бодовног система и оцени интерфејса према успостављеним принципима корисничког дизајна, након чега се излаже инструмент припремљен за испитивање са корисницима. Поглавље се завршава прегледом ограничења спроведеног вредновања.

9.1. Вредновање компоненте вештачке интелигенције

Испитивање је обухватило 430 позива упућених ка пет језичких модела доступних преко истог сучеља. Модели су изабрани тако да покрију распон од великих комерцијалних до мањих отворених модела. Сви су позивани са истим упитима и истим поставкама. Обим по моделу наведен је у табели 9.1.1.

Табела 9.1.1. Испитани језички модели и обим спроведених мерења

Модел	Број позива	Обим покретања
Phi-4	110	потпуно
Meta-Llama-3.1-8B-Instruct	110	потпуно
Mistral-small-2503	110	потпуно
gpt-4o-mini	60	делимично
gpt-4o	20	делимично
Mistral-small	20	неуспело

Три покретања обављена су у пуном обиму, док су два прекинута пре завршетка због ограничења броја позива које сервис намеће. Шесто покретање у потпуности је отказало и о њему се посебно расправља у одељку 9.1.3. Резултати изложени у наставку рачунати су на 410 успешно обрађених позива. Испитана су три модула: препоручивач, генератор и тутор, док модул контекстуалне помоћи није обухваћен посебним мерењем због тога што његови одговори немају проверљив тачан исход.

9.1.1. Тачност препоруке и проблем изједначених резултата

Модул препоручивача добија табелу са бројем проширених чворова за свих осам алгоритама на једној мапи и треба да наведе који је обрадио најмање, а који највише чворова. Задатак је намерно постављен као читавање из табеле, јер тако мери способност нумеричког закључивања, за коју је познато да представља слабост језичких модела [33].

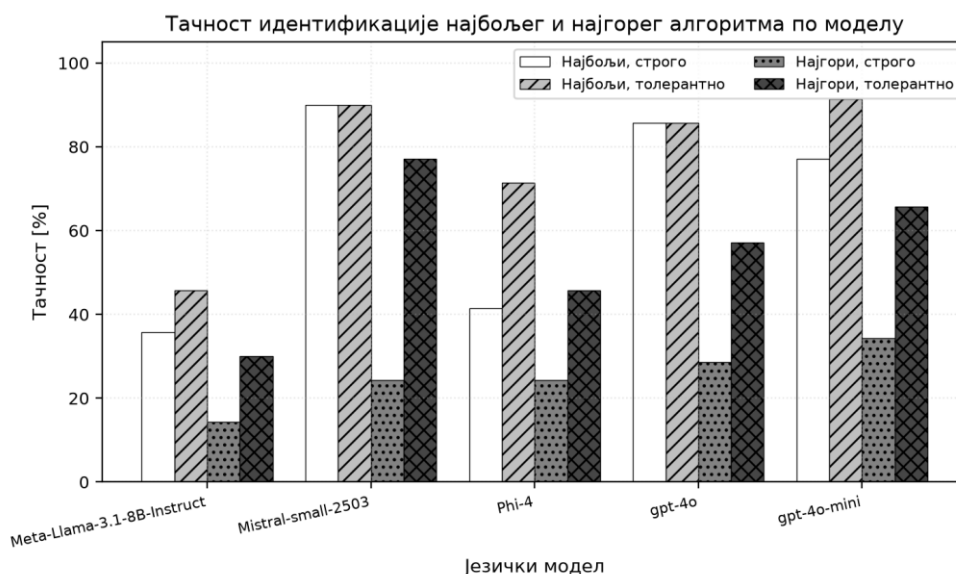
Прва обрада података примењивала је строгу меру, то јест поређење одговора са забележеном ознаком стварног екстрема, а тако добијене вредности биле су изненађујуће ниске. Пажљивијим прегледом уочен је методолошки недостатак. У великом броју случајева више алгоритама има потпуно исти број проширених чворова, што је очекивано на мапама

где хеуристика нема утицаја. Строга мера у таквим случајевима проглашава одговор нетачним чак и када наведени алгоритам има управо најмању или највећу вредност, само зато што је записана ознака припала неком другом алгоритму са истом вредношћу. У 61,90 одсто случајева најмању вредност деле бар два алгоритма, а у 64,29 одсто то важи и за највећу, чиме се обим појаве показује знатним. Због тога је уведена толерантна мера тачности, према којој се одговор прихвата као тачан ако наведени алгоритам има вредност једнаку екстремној. Ради потпуне провере, вредности из табеле поново су прочитане из текста самог упита и упоређене са забележеним ознакама, а несагласни позиви искључени су из анализе.

Табела 9.1.2 и слика 9.1.1 приказују тачност по моделима према обема мерама.

Табела 9.1.2. Тачност идентификације најбољег и најгорег алгоритма по моделу

Модел	Покретање	N (укупно)	N (препоруча)	Најбољи тачно (%)	Најбољи тачно, толерантно (%)	Најгори тачно (%)	Најгори тачно, толерантно (%)	Исправан JSON (%)	Медијана латенције (ms)
Meta-Llama-3.1-8B-Instruct	потпуно	110	70	35,71	45,71	14,29	30,00	99,09	1102,75
Mistral-small-2503	потпуно	110	70	90,00	90,00	24,29	77,14	100,00	1420,45
Phi-4	потпуно	110	70	41,43	71,43	24,29	45,71	100,00	3106,10
gpt-4o	делимично	20	7	85,71	85,71	28,57	57,14	100,00	1945,20
gpt-4o-mini	делимично	60	35	77,14	91,43	34,29	65,71	100,00	1614,45
Mistral-small	неуспело	20	7	0,00	0,00	0,00	0,00	0,00	246,40



Слика 9.1.1. Тачност идентификације најбољег и најгорег алгоритма по моделу

Разлика између двеју мера је значајна. За најгори алгоритам укупна тачност по строгој мери износи 23,02 одсто, а по толерантној 53,17 одсто, више од двоструко. За најбољи алгоритам вредности износе 59,52 наспрам 72,62 одсто. Овај налаз показује да мера тачности примењена без разматрања структуре података може довести до потпуно погрешне слике о учинку модела.

Друга група налаза односи се на разлике међу моделима. Најбољи резултат по строгој мери постиже модел Mistral-small-2503 са 90,00 одсто за најбољи алгоритам, што надмашује

и модел gpt-4o који постиже 85,71 одсто. Овај резултат треба тумачити уз опрез, зато што је покретање модела gpt-4o обухватило само седам позива у овом модулу. Чак и уз ту ограду он показује да мањи и јефтинији модели у уско постављеном задатку могу бити подједнако употребљиви као већи. Модел Meta-Llama-3.1-8B-Instruct јасно заостаје са 35,71 одсто по строгој мери, што одговара очекивању да способност рада са бројевима расте са величином модела. Занимљива је и разлика код модела Phi-4, где тачност расте са 41,43 на 71,43 одсто. Тај модел најчешће наводи алгоритам који јесте међу најбољима, а греша само у избору међу једнако добрим одговорима.

9.1.2. Генератор сценарија и тутор

Модул генератора испитан је на десет унапред састављених захтева поновљених за сваки модел, укупно на педесет позива, а модул тутора на 108 позива. Резултати су приказани у табели 9.1.3.

Табела 9.1.3. Резултати модула генератора и тутора

Мерена величина	Генератор	Тутор
број позива	50	108
исправно обликован одговор	100,00 %	99,07 %
успешно издвојена намера	100,00 %	није примењиво
захтев задовољен	66,00 %	није примењиво
одговор са важећим тренуцима	није примењиво	99,07 %
просечан број враћених тренутака	није примењиво	3,00 од 3
медијана времена одзива	1660 ms	2590 ms

Раздвајање издвајања намере од задовољења захтева представља најважнији налаз модула генератора. Разумевање тога шта корисник тражи успело је у свим случајевима без изузетка, док је проналажење мапе која заиста производи тражено понашање успело у две трећине случајева. Прегледом појединачних случајева утврђено је да неуспеси нису равномерно распоређени. Захтеви попут проналажења мапе на којој је алгоритам А* знатно бржи од Дајкстриног алгоритма увек су задовољени, док захтеви попут истицања претраге у ширину наспрам свих осталих поступака доследно нису. Објашњење произлази непосредно из резултата претходног поглавља. Претрага у ширину никада не обрађује најмањи број чворова, па мапа са траженим својством вероватно и не постоји у класи мапа које систем уме да генерише. Неуспех стога не потиче од језичког модела, већ од ограничења скупа генератора. Практична последица је непосредна: проширење тог скупа повећало би удео задовољених захтева више него замена језичког модела.

Модул тутора показао се као најпоузданији. То је последица пројектне одлуке описане у одељку 6.2: модел не бира тренутке, већ само објашњава оне које је програм издвојио. Простор за грешку тиме је сведен на обликовање одговора. Просечан број враћених тренутака износи тачно три од три тражена, што показује да модели доследно поштују задату структуру. Овај резултат представља најјаснију потврду општег приступа изложеног у поглављу 6. Модул у коме је улога модела најуже дефинисана истовремено је и најпоузданији, док модул који од модела тражи нумеричко закључивање показује највећу променљивост.

9.1.3. Време одзива и робусност

Времена одзива крећу се у распону од 646 милисекунди до 14,6 секунди, а медијана износи око 1,7 секунди. Модул тутора има највеће време одзива јер тражи најдужи одговор.

Модел Meta-Llama-3.1-8B-Instruct има медијану од 1103 милисекунде, а модел Phi-4 медијану од 3106 милисекунди, готово три пута већу, што разлике међу моделима чини значајнијим од разлика међу модулима. У наставном контексту, где корисник чека одговор непосредно после интеракције, ова разлика је приметна и представља ваљан критеријум при избору модела, поред тачности. Удео исправно обликованих одговора код успешних покретања износи 99,76 одсто, то јест један неисправан одговор на 410 позива. Тиме је потврђено да тражење одговора у машински читљивом облику уз опис очекиване структуре представља поуздан приступ.

Посебну пажњу заслужује шесто покретање из табеле 9.1.1, у коме ниједан од двадесет позива није враћен у исправном облику. Узрок је утврђен као погрешно наведена ознака модела, јер сервис такву ознаку не препознаје и враћа поруку о грешци уместо одговора. Овај случај, иако настао ненамерно, представља користан налаз. Он показује да отказ спољашњег сервиса не мора бити повремени и појединачан, већ може бити потпун и трајан. Систем је такав отказ поднео без последица, зато што је свака обрада одговора обавијена провером, па је кориснику приказана порука о грешци. Мерење времена одзива у том покретању, са медијаном од 246 милисекунди, показује да су одговори стизали брзо, што уједно значи да се кратко време одзива не сме тумачити као показатељ доброг рада.

Резултати изложени у овом одељку поткрепљују пројектну одлуку из поглавља 6, према којој језички модел не сме давати податак који систем може сам да израчуна. Тачност у задатку читавања вредности из табеле креће се између 35,71 и 90,00 одсто. Пошто систем све бројчане тврдње изводи из стварног покретања алгоритама, ова несигурност уопште не долази до корисника. Методолошки налаз о потцењивању учинка при изједначеним вредностима применљив је и изван овог рада, пошто задаци у којима модел треба да изабере екстрем из скупа често садрже изједначене вредности.

9.2. Верификација бодовног система

Бодовни систем описан у одељку 5.4 представља средство помоћу кога систем даје повратну информацију о квалитету корисничког решења. Да би та информација имала смисла, бодовање мора бити исправно, праведно и осетљиво на тежину задатка. Пошто провера на стварним корисницима захтева велики број покушаја, спроведена је аутоматизована симулација седам типова играча на стотинама мапа, чиме је добијено 11250 покушаја. Резултати су приказани у табели 9.2.1.

Табела 9.2.1. Расподела поена по типу симулираног играча

Тип играча	N	Просек	Медијана	Ст. девијација	Мин	Макс	Пенал за цену	Пенал за грешке
савршен	2250	106,62	110,00	12,54	60	110	3,38	0,00
добар	2250	98,49	101,00	12,52	55	110	7,10	0,00
слаб	2250	60,35	50,00	20,77	50	110	40,00	0,00
неисправни потези	2250	95,61	100,00	14,99	40	110	3,33	9,85
тачно „нема пута”	750	108,00	108,00	0,00	108	108	0,00	0,00
погрешно „нема пута”	750	0,00	0,00	0,00	0	0	50,00	0,00
пут на нерешивој мапи	750	0,00	0,00	0,00	0	0	50,00	50,00

Просечан број поена монотono опада од савршеног, преко доброг, до слабог играча, са 106,6 на 98,5 па на 60,3 поена, чиме резултати потврђују очекивано понашање. Постоје два гранична случаја: корисник погрешно закључује да пут не постоји или предаје пут на нерешивој мапи. Оба доследно дају нула поена у свих 750 покушаја по типу. Тачно препознавање нерешиве мапе доследно даје 108 поена.

Две појаве захтевају објашњење. Прва је да савршен играч не добија највећи могући број поена у свих сто одсто случајева, већ у 93,2 одсто. Разлог је у томе што се на мапама са пондерисаним тереном као референтни алгоритам понекад користи претрага у ширину, чији пут није најјефтинији. Друга је висок просечан број поена играча који чини неисправне потезе, 95,6. Симулација прескаче свега три ћелије и производи само један неисправан потез. То показује ограничење симулације, а не бодовања.

Осетљивост на тежину задатка проверена је поређењем поена играча означеног као добар по типовима мапа. Крускал-Волисовим тестом утврђено је да разлике међу типовима нису случајне: $H = 616,82$ уз $p < 0,001$ за седам група. Најнижи резултати добијени су на мапама са пондерисаним тереном, 90,0 поена. То су мапе на којима избор пута заиста утиче на цену. Највиши резултати добијени су на мапама типа лавиринт, 105,0 поена, што одговара налазу из одељка 8.2 према коме у лавиринту постоји само један пут. Овај налаз потврђује да бодовање одражава стварну тежину задатка, али уједно указује и на то да лавиринт као тип мапе за ову вежбу није погодан.

9.3. Вредновање корисничког интерфејса

Интерфејс је оцењен према четири успостављена скупа принципа: Нилсеновим хеуристикама [24, 25], Тогазинијевим принципима интеракције [26], Шнајдермановим златним правилима [27] и Нормановим принципима дизајна [28]. Оцењивање је спровео аутор рада, што представља ограничење о коме се расправља у одељку 9.5. Табела 9.3.1 сажима оцене по Нилсеновим хеуристикама, које чине најшире прихваћен оквир.

Табела 9.3.1. Оцене према Нилсеновим хеуристикама

Хеуристика	Оцена	Најзначајније запажање
видљивост стања система	испуњено	државна машина, бројач корака, мерене величине у стварном времену
подударање са стварним светом	испуњено	мрежа као мапа, боје са природним значењем, два језика
контрола и слобода корисника	делимично	потпуно управљање анимацијом, али нема поништавања предаје
доследност и стандарди	испуњено	иста боја за исти алгоритам и исти распоред на све три странице
спречавање грешака	испуњено	заштита полазног и циљног чвора, онемогућена дугмад, провера пута
препознавање уместо присећања	испуњено	стално видљива легенда и мерене величине
прилагодљивост и ефикасност	испуњено	пречице за искусне уз водич за почетнике
естетика и минимализам	испуњено	напредне поставке скривене у падајуће меније
препознавање и опоравак од грешака	делимично	поруке о грешкама јасне, али мрежне грешке уопштене

Хеуристика	Оцена	Најзначајније запажање
помоћ и документација	испуњено	водич од осамнаест корака и објашњења на захтев

Две ставке оцењене су као делимично испуњене. Након предаје решења у режиму полигона враћање није могуће, што је уведено намерно како би се спречило да корисник понављањем дође до решења, а за мрежне грешке приказује се уопштена порука.

Преостала три скупа принципа у великој мери се преклапају са Нилсеновим, па се овде издвајају само ставке које уносе нову перспективу. Тогазинијев принцип предвиђања остварен је аутоматским прилагођавањем контрола изабраном алгоритму, а принцип смањења кашњења претходним израчунавањем целог трага. Шнајдерманово правило о дијалозима са јасним завршетком остварено је у сваком од три режима, а Норманов принцип пресликавања просторном подударношћу између мреже на екрану и мапе коју она представља. Два недостатка заједничка су свим оквирима, а то су изостанак општег механизма за поништавање низа радњи и то што се ручно нацртана мапа која није сачувана губи при освежавању странице.

9.4. Припремљени инструмент за корисничко тестирање

Хеуристичка евалуација представља експертску процену и не замењује испитивање са стварним корисницима. За такво испитивање припремљен је инструмент чији је потпун текст дат у прилогу В. Оно у оквиру овог рада није спроведено, па се овде излаже само његов састав и начин обраде добијених података.

Инструмент обухвата шест задатака обликованих тако да покрију основне токове рада у укупном трајању од око двадесет минута. Први задатак проверава цртање мапе и покретање визуелизације, други читавање стања из приказа, а трећи разумевање разлике између броја корака и цене пута у режиму поређења. Четврти задатак обухвата рад у режиму полигона, пети употребу генератора на природном језику, а шести чување и учитавање мапе. За сваки задатак предвиђено је бележење времена решавања, исхода у три степена: решен без помоћи, решен уз помоћ или нерешен, као и запажања изречених наглас током рада.

Након решавања задатака предвиђено је попуњавање стандардног упитника за процену употребљивости [29], који садржи десет тврдњи са петостепеном скалом слагања. Изабран је зато што је кратак, широко примењен и зато што постоје објављене одреднице за тумачење добијене оцене. Према тим одредницама, вредност изнад 68 сматра се натпросечном, вредност изнад 80,3 сврстава систем у горњих десет одсто, а вредност испод 51 указује на озбиљне тешкоће у употреби [30]. Обрада би обухватила успешност по задацима, просек, медијану, стандардну девијацију и 95 одсто интервал поверења укупне оцене, рачунат изразом 7.3.1. Испитаници би били раздвојени према томе да ли су слушали предмет на коме се обрађују алгоритми над графовима.

Величина узорка не мора бити велика да би испитивање било корисно, јер је показано да већ пет испитаника открије око 80 одсто постојећих проблема употребљивости [25]. Спровођење овог испитивања наведено је као правац даљег рада у одељку 10.2.

9.5. Ограничења спроведеног вредновања

Приликом тумачења резултата овог поглавља треба имати у виду четири ограничења. Прво, испитивање са стварним корисницима није спроведено, већ је само припремљен инструмент из одељка 9.4. Хеуристичку евалуацију спровео је аутор система, док литература

препоручује између три и пет независних оцењивача. Један оцењивач открива око 35 одсто постојећих проблема употребљивости [24]. Због тога оцене из одељка 9.3 треба разумети као систематску самопроцену, а не као налаз о томе како се стварни корисници сналазе у систему. Друго, верификација бодовног система спроведена је симулацијом, која поуздано проверава да ли се бодовање понаша према сопственој дефиницији, али не може да утврди да ли добијена оцена одговара интуитивном осећају корисника. Треће, систем није јавно постављен, па није било могуће прикупити податке о употреби током дужег периода, а разлози за ту одлуку разматрани су у одељку 10.2. Четврто, испитивање дугорочног утицаја на исходе учења није спроведено, јер би захтевало контролисани експеримент са контролном групом, што излази изван обима овог рада.

Треба напоменути и да је сервис преко кога су језички модели позивани повучен након спровођења мерења, због чега поновно извођење тог дела експеримента захтева други извор истих или упоредивих модела.

Ово поглавље показало је да је усвојена подела одговорности у слоју вештачке интелигенције оправдана мерењем. Показано је и да бодовни систем ради исправно и да је осетљив на тежину задатка. Интерфејс испуњава већину успостављених принципа корисничког дизајна, уз идентификоване недостатке који се односе на поништавање радњи и на обраду мрежних грешака. Потврда тих налаза са стварним корисницима остаје за даљи рад. Наредно поглавље сумира доприносе рада.

10. ЗАКЉУЧАК

Задатак постављен на почетку рада био је да се пројектује и реализује интерактивна веб апликација која повезује три целине. То су визуелизација алгоритама за проналажење пута, инфраструктура за поновљива мерења и слој заснован на великим језичким моделима. Те три целине се у постојећим решењима ретко срећу заједно. Реализовани систем обухвата осам алгоритама, три режима рада и седам генератора мапа. Обухвата и потпуну аутентикацију корисника, трајно чување мапа и резултата, подршку за два језика и четири модула вештачке интелигенције. Постављени задатак тиме је испуњен у целости.

Најзначајније пројектно решење представља потпуно раздвајање алгорита од приказа, остварено кроз јединствени интерфејс државне машине и стандардизован систем од осам врста догађаја. Захваљујући томе, приказ не садржи ниједну грану условљену врстом алгорита, а пет од осам поступака реализовано је кроз једну машину претраге код које се разлика своди на једну методу за израчунавање приоритета. Ово решење не само да смањује обим кода, већ и сужава простор у коме треба тражити узрок неисправног понашања, што се показало корисним током развоја.

Експериментални део обухватио је 9444 симулације алгоритама, 11250 симулација бодовног система и 410 позива упућених ка пет језичких модела. Уведене су две мере које уобичајена излагања ове материје не садрже. Прва је заједничка скала цене пута, добијена мерењем цене на путу који је алгоритам вратио уместо преузимања интерне вредности алгорита, чиме је омогућено поређење поступака који растојање дефинишу на различите начине. Друга је мера субоптималности, дефинисана као релативно одступање од доказано најјефтинијег пута, која поређење чини независним од димензија и густине мапе.

Мерења су дала три налаза која заслужују истраживање. Претрага у ширину показала је субоптималност од 18,11 одсто, што њену гаранцију проналажења најкраћег пута ставља у прави контекст, јер она минимизује број корака, а не цену. Претрага у дубину показала је највећи меморијски отисак од свих поступака, 145,49 чворова у просеку наспрам 29,33 колико их има претрага у ширину. Тај налаз супротан је закључку који се изводи анализом стабла претраге. Разлог је у томе што се на графу на стек одлажу сви непосећени суседи целог текућег пута. Претрага 0-1 у ширину, примењена на мапе са тежинама изван скупа за који је намењена, показала је субоптималност од 128,51 одсто уз потпуно исправну реализацију. Сва три налаза поткрепљују тврдњу да својства алгоритама важе само унутар модела за који су изведена, а мерење ту тврдњу претвара из формалне напомене у бројку.

Испитивање језичких модела дало је два доприноса. Први допринос је архитектонски: подела по којој програм рачуна, а модел објашњава. Мерење је ту поделу оправдало. Тачност модела у задатку читавања екстремне вредности из табеле кретала се између 35,71 и 90,00 одсто. Такав распон не допушта да се одговор модела прикаже кориснику као чињеница. Други допринос је методолошки и односи се на уочавање да строга мера тачности потцењује учинак модела онда када подаци садрже изједначене вредности. У више од шездесет одсто мерења најмању вредност делило је више алгоритама. Због тога је уведена толерантна мера,

која је тачност за најгори алгоритам подигла са 23,02 на 53,17 одсто. Та разлика довољно је велика да би закључак изведен само на основу строге мере био погрешан.

Основна погодност реализованог решења јесте то што визуелизацију, мерење и активно учешће корисника обједињује у једном алату, чиме постаје употребљиво и у настави и у систематској анализи. Главна ограничења произлазе из обима рада. Систем ради у локалном окружењу. Испитивање са стварним корисницима није спроведено, већ је само припремљен инструмент за њега. Вредновање корисничког интерфејса засновано је на самопроцени уместо на независним оцењивачима, а утицај на исходе учења није непосредно измерен. Класа мапа на којима су мерења спроведена одређена је реализованим генераторима, па се закључци не морају пренети на мапе битно другачијих својстава. Подручје примене обухвата наставу из алгоритама и структура података, самостално учење, као и припрему наставних материјала, пошто систем омогућава да наставник унапред припреми и подели скуп мапа за вежбу.

10.1. Правци даљег развоја

Проблеми уочени током рада отварају неколико праваца за даље проширење решења.

У погледу алгоритама, систем би се могао проширити поступцима прилагођеним мрежним мапама. Пример је алгоритам Jump Point Search [13], који прескакањем чворова постиже вишеструко убрзање на униформним мрежама. Други пример је поступак Theta* [14], који напушта ограничење кретања по ивицама мреже. Посебно је занимљиво увођење поступака за динамичко поновно планирање, попут алгоритма D* Lite [15], уз мерење броја поновних израчунавања при изменама мапе током кретања. Овај правац био је предвиђен почетном замисли пројекта, али је изостављен ради усредсређивања на дубину обраде постојећег обима.

У погледу мерења, употреба стандардизоване збирке мапа [16] омогућила би поређење добијених резултата са објављеним резултатима других аутора, док би допуна постојећих мера бројањем основних операција поређење учинила потпуно независним од извршног окружења.

У погледу компоненте вештачке интелигенције, налаз из одељка 9.1 указује да би проширење скупа генератора мапа повећало удео задовољених захтева више него замена језичког модела. Занимљив правац представља и испитивање да ли подстицање модела на поступно закључивање [34] побољшава тачност у задатку читавања из табеле, јер је познато да тај поступак помаже управо код нумеричких задатака.

У погледу вредновања, први корак јесте спровођење испитивања са стварним корисницима према инструменту из прилога В, што би хеуристичку оцену из одељка 9.3 потврдило или оповргло. Најзначајнији недостајући део јесте контролисани експеримент са контролном групом, којим би се измерио утицај употребе система на исходе учења. Јавно постављање система омогућило би прикупљање података о употреби током дужег периода, уз потребна ограничења броја позива по кориснику.

Систем описан у овом раду показао је да визуелизација, мерење и активно учешће корисника могу бити обједињени у једном алату. Показао је и да савремени језички модели у таквом алату имају употребљиву улогу, под условом да им се она прецизно ограничи. Мерења спроведена у оквиру рада уједно су показала да и добро познати алгоритми, када се испитају систематски, откривају појаве које уобичајено излагање материје не обухвата.

ЛИТЕРАТУРА

- [1] Cormen Thomas, Leiserson Charles, Rivest Ronald, Stein Clifford, „Introduction to Algorithms,” четврто издање, MIT Press, Кембриџ, 2022, стр. 594–642.
- [2] Russell Stuart, Norvig Peter, „Artificial Intelligence: A Modern Approach,” четврто издање, Pearson, Хобокен, 2021, стр. 71–116.
- [3] Dijkstra Edsger, „A note on two problems in connexion with graphs,” *Numerische Mathematik*, том 1, свеска 1, 1959, стр. 269–271.
- [4] Hart Peter, Nilsson Nils, Raphael Bertram, „A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Transactions on Systems Science and Cybernetics*, том 4, свеска 2, 1968, стр. 100–107.
- [5] Hart Peter, Nilsson Nils, Raphael Bertram, „Correction to A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *ACM SIGART Bulletin*, свеска 37, 1972, стр. 28–29.
- [6] Pohl Ira, „Heuristic search viewed as path finding in a graph,” *Artificial Intelligence*, том 1, свеска 3–4, 1970, стр. 193–204.
- [7] Pearl Judea, „Heuristics: Intelligent Search Strategies for Computer Problem Solving,” Addison-Wesley, Ридинг, 1984, стр. 73–110.
- [8] Likhachev Maxim, Gordon Geoffrey, Thrun Sebastian, „ARA*: Anytime A* with Provable Bounds on Sub-Optimality,” *Advances in Neural Information Processing Systems*, том 16, 2003, стр. 767–774.
- [9] Ebendt Rüdiger, Drechsler Rolf, „Weighted A* search, unifying view and application,” *Artificial Intelligence*, том 173, свеска 14, 2009, стр. 1310–1342.
- [10] Ahuja Ravindra, Magnanti Thomas, Orlin James, „Network Flows: Theory, Algorithms, and Applications,” прво издање, Prentice Hall, Енглвуд Клифс, 1993, стр. 108–116.
- [11] Dial Robert, „Algorithm 360: Shortest-path forest with topological ordering,” *Communications of the ACM*, том 12, свеска 11, 1969, стр. 632–633.
- [12] Patel Amit, „Introduction to the A* Algorithm,” Red Blob Games, <https://www.redblobgames.com/pathfinding/a-star/introduction.html>, приступано јул 2026.
- [13] Harabor Daniel, Grastien Alban, „Online Graph Pruning for Pathfinding on Grid Maps,” *Proceedings of the 25th AAAI Conference on Artificial Intelligence*, Сан Франциско, 2011, стр. 1114–1119.
- [14] Nash Alex, Daniel Kenny, Koenig Sven, Felner Ariel, „Theta*: Any-Angle Path Planning on Grids,” *Proceedings of the 22nd AAAI Conference on Artificial Intelligence*, Ванкувер, 2007, стр. 1177–1183.
- [15] Koenig Sven, Likhachev Maxim, „D* Lite,” *Proceedings of the 18th AAAI Conference on Artificial Intelligence*, Едмонтон, 2002, стр. 476–483.

- [16] Sturtevant Nathan, „Benchmarks for Grid-Based Pathfinding,” IEEE Transactions on Computational Intelligence and AI in Games, том 4, свеска 2, 2012, стр. 144–148.
- [17] Hundhausen Christopher, Douglass Sarah, Stasko John, „A Meta-Study of Algorithm Visualization Effectiveness,” Journal of Visual Languages and Computing, том 13, свеска 3, 2002, стр. 259–290.
- [18] Naps Thomas, Rößling Guido, Almstrum Vicki и остали, „Exploring the Role of Visualization and Engagement in Computer Science Education,” ACM SIGCSE Bulletin, том 35, свеска 2, 2002, стр. 131–152.
- [19] Shaffer Clifford, Cooper Matthew, Alon Alexander и остали, „Algorithm Visualization: The State of the Field,” ACM Transactions on Computing Education, том 10, свеска 3, 2010, чланак 9.
- [20] Brown Marc, Sedgewick Robert, „A system for algorithm animation,” ACM SIGGRAPH Computer Graphics, том 18, свеска 3, 1984, стр. 177–186.
- [21] Halim Steven, Halim Felix, Koh Zi Chun, „VisuAlgo: visualising data structures and algorithms through animation,” National University of Singapore, <https://visualgo.net/en>, приступано јул 2026.
- [22] Mihailescu Clément, „Pathfinding Visualizer,” 2017, <https://github.com/clementmihailescu/Pathfinding-Visualizer>, приступано јул 2026.
- [23] Xu Xueqiao, „PathFinding.js: A comprehensive path-finding library for grid based games,” 2011, <https://github.com/qiao/PathFinding.js>, приступано јул 2026.
- [24] Nielsen Jakob, Molich Rolf, „Heuristic evaluation of user interfaces,” Proceedings of the ACM CHI Conference on Human Factors in Computing Systems, Сијетл, 1990, стр. 249–256.
- [25] Nielsen Jakob, „Enhancing the explanatory power of usability heuristics,” Proceedings of the ACM CHI Conference on Human Factors in Computing Systems, Бостон, 1994, стр. 152–158.
- [26] Tognazzini Bruce, „First Principles of Interaction Design,” <https://asktog.com/atc/principles-of-interaction-design/>, приступано јул 2026.
- [27] Shneiderman Ben, Plaisant Catherine, Cohen Maxine, Jacobs Steven, Elmqvist Niklas, „Designing the User Interface: Strategies for Effective Human-Computer Interaction,” шесто издање, Pearson, Бостон, 2016, стр. 88–96.
- [28] Norman Donald, „The Design of Everyday Things,” допуњено издање, Basic Books, Њујорк, 2013, стр. 10–36.
- [29] Brooke John, „SUS: A quick and dirty usability scale,” у зборнику Usability Evaluation in Industry, Taylor and Francis, Лондон, 1996, стр. 189–194.
- [30] Bangor Aaron, Kortum Philip, Miller James, „An Empirical Evaluation of the System Usability Scale,” International Journal of Human-Computer Interaction, том 24, свеска 6, 2008, стр. 574–594.
- [31] Vaswani Ashish, Shazeer Noam, Parmar Niki и остали, „Attention Is All You Need,” Advances in Neural Information Processing Systems, том 30, 2017, стр. 5998–6008.

- [32] Brown Tom, Mann Benjamin, Ryder Nick и остали, „Language Models are Few-Shot Learners,” Advances in Neural Information Processing Systems, том 33, 2020, стр. 1877–1901.
- [33] Frieder Simon, Pinchetti Luca, Griffiths Ryan-Rhys и остали, „Mathematical Capabilities of ChatGPT,” Advances in Neural Information Processing Systems, том 36, 2023, стр. 27699–27744.
- [34] Wei Jason, Wang Xuezhi, Schuurmans Dale и остали, „Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” Advances in Neural Information Processing Systems, том 35, 2022, стр. 24824–24837.
- [35] Google, „Angular documentation,” издање 21.2, <https://angular.dev/>, приступано јул 2026.
- [36] OpenJS Foundation, „Express: Fast, unopinionated, minimalist web framework for Node.js,” издање 5.2, <https://expressjs.com/>, приступано јул 2026.
- [37] MongoDB Inc., „Mongoose ODM documentation,” издање 9.4, <https://mongoosejs.com/docs/>, приступано јул 2026.
- [38] OpenJS Foundation, „Socket.IO documentation,” издање 4.8, <https://socket.io/docs/v4/>, приступано јул 2026.
- [39] Tailwind Labs, „Tailwind CSS documentation,” издање 4.2, <https://tailwindcss.com/docs>, приступано јул 2026.
- [40] GitHub Inc., „GitHub Models documentation,” <https://docs.github.com/en/github-models>, приступано јул 2026.

СПИСАК СКРАЋЕНИЦА

API	Application Programming Interface
ARA*	Anytime Repairing A*
BFS	Breadth-First Search
CLI	Command Line Interface
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DFS	Depth-First Search
DOM	Document Object Model
GPT	Generative Pre-trained Transformer
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDA*	Iterative Deepening A*
JPS	Jump Point Search
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
ODM	Object Document Mapper
REST	Representational State Transfer
SUS	System Usability Scale
UML	Unified Modeling Language

СПИСАК СЛИКА

Слика 4.1.1. Слојевита архитектура система.....	10
Слика 5.1.1. Заједнички распоред екрана у свим режимима	14
Слика 5.2.1. Аутоматско генерисање мапе.....	15
Слика 5.2.2. Ток претраге у режиму визуелизације.....	16
Слика 5.3.1. Упоредни приказ рада више алгоритама.....	17
Слика 5.4.1. Конструисање пута у режиму полигона.....	18
Слика 5.4.2. Оцењивање корисничког решења у режиму полигона	19
Слика 5.5.1. Интерактивни водич кроз апликацију	19
Слика 5.5.2. Кориснички профил и статистика.....	20
Слика 6.1.1. Ток обраде захтева у слоју вештачке интелигенције	22
Слика 6.2.1. Панел тутора са објашњењима кључних тренутака.....	23
Слика 6.4.1. Панел препоручивача алгоритма	24
Слика 8.2.1. Број проширених чворова по типу мапе	30
Слика 8.2.2. Утицај густине препрека на број проширења и на решивост мапа	31
Слика 8.3.1. Број проширених чворова у односу на величину мреже	32
Слика 8.5.1. Утицај параметра w на цену претраге и квалитет пута	34
Слика 8.7.1. Компромис између цене претраге и квалитета пута	36
Слика 9.1.1. Тачност идентификације најбољег и најгорег алгоритма по моделу	38

СПИСАК ТАБЕЛА

Табела 2.6.1. Својства хеуристичких функција	4
Табела 2.7.1. Упоредни преглед разматраних алгоритама.....	5
Табела 3.2.1. Упоредни преглед постојећих решења	7
Табела 3.3.1. Преглед технолошких избора и разлога	8
Табела 4.2.1. Врсте догађаја које емитују алгоритми.....	11
Табела 4.5.1. Реализовани генератори мапа	12
Табела 4.6.1. Колекције у бази података.....	13
Табела 6.5.1. Подела одговорности у модулима вештачке интелигенције	24
Табела 7.1.1. Мерење величине и њихово значење	26
Табела 7.2.1. Категорије спроведених мерења	27
Табела 8.1.1. Збирни преглед свих алгоритама на мрежама димензија 25×50	29
Табела 8.2.1. Субоптималност алгоритама по типу мапе, у процентима	30
Табела 8.3.1. Емпиријски експонент раста броја проширених чворова	32
Табела 8.4.1. Број проширења по изабраној хеуристичкој функцији	32
Табела 8.4.2. Поређење суседства са четири и са осам суседа	33
Табела 8.5.1. Утицај параметра w на број проширења и на квалитет пута	34
Табела 8.6.1. Максимална величина фронтијера по алгоритму	35
Табела 9.1.1. Испитани језички модели и обим спроведених мерења.....	37
Табела 9.1.2. Тачност идентификације најбољег и најгорег алгорита по моделу.....	38
Табела 9.1.3. Резултати модула генератора и татора	39
Табела 9.2.1. Расподела поена по типу симулираног играча.....	40
Табела 9.3.1. Оцене према Нилсеновим хеуристикама	41

ПРИЛОГ А: СТРУКТУРА ПРОЈЕКТА И УПУТСТВО ЗА ПОКРЕТАЊЕ

Изворни код организован је у три главна директоријума. Клијентски и серверски део међусобно су независни, а директоријум са заједничким типовима користе оба.

```
Имплементација/  
├── client/src/app/  algorithms, components, pages, services, generators, guards  
├── server/src/      routes, services, models, middleware, run-benchmark.ts,  
                    run-ai-benchmark.ts  
└── shared/types/    заједнички типови за клијент и сервер  
  
Метрике/  
├── Алгоритми/, AI/, Playground/  изворни подаци мерења  
├── Анализа/analiza.py             статистичка обрада и цртање графика  
└── Сlike/                         графички прикази коришћени у раду
```

А.1. Захтеви, подешавање и покретање

За покретање је потребно окружење Node.js издања 24 или новијег и приступ бази података MongoDB. Поставке се задају у датотеци `.env` у директоријуму `server`, у коју се уносе адреса базе података, тајна вредност за потписивање токена, приступни подаци за сервис за складиштење слика и приступни кључ за језичке моделе. Зависности се инсталирају наредбом `npm install`, засебно у сваком од двају директоријума.

Серверски део покреће се из директоријума `server` наредбом `npm run dev`, а клијентски из директоријума `client` наредбом `npm start`, након чега је апликација доступна на подразумеваној адреси локалног сервера. Мерење алгоритама покреће се наредбом `npx ts-node src/run-benchmark.ts`, уз могућност навођења појединачних категорија, чиниоца умножавања броја мапа заставицом `--scale` и рада без базе података заставицом `--no-db`. Мерење језичких модела покреће се наредбом `npx ts-node src/run-ai-benchmark.ts`, а статистичка обрада наредбом `python Metrike/Analiza/analiza.py`.

А.2. Формати улазних и излазних датотека

Мапе се чувају у бази у документном облику који садржи димензије мреже, положаје полазног и циљног чвора, списак зидова и списак ћелија са повећаном тежином. Резултати мерења извозе се у формату CSV, са по једним редом за сваку симулацију. Колоне описују категорију мерења, поставке генератора, поставке алгорита и све мерене величине наведене у одељку 7.1. Исти подаци извозе се и у формату JSON, у структурираном облику. Резултати појединачног покретања у корисничком интерфејсу извозе се у оба формата преко дугмади у површини са мереним величинама.

ПРИЛОГ Б: НАЈВАЖНИЈИ ДЕЛОВИ ПРОГРАМСКОГ КОДА

У овом прилогу наведени су делови кода који су у тексту рада поменути, али нису у целости приказани. Наводе се само делови суштински за разумевање решења, док је потпун изворни код доступан уз рад.

Б.1. Јединствени интерфејс алгоритма

Сваки од осам алгоритама реализује интерфејс приказан у наставку, чиме је постигнуто раздвајање описано у одељку 4.2. Метода за корак извршава једну логичку операцију и враћа списак насталих догађаја, што омогућава да приказ буде потпуно независан од врсте алгоритма.

```
export interface PathfindingAlgorithm {
  init(grid: Grid, start: Position, goal: Position,
    options: AlgorithmOptions): void;
  step(): AlgorithmEvent[];
  isDone(): boolean;
  getResult(): AlgorithmResult | null;
  getTrace(): AlgorithmEvent[];
}
```

Б.2. Хеуристичке функције и цена прелаза

Све четири хеуристике реализоване су у једној функцији, чиме се позивно место не мења при промени избора. Октилна хеуристика записана је у облику који је алгебарски једнак изразу 2.6.4, али захтева једно рачунање мање. Функција за цену прелаза одговара изразу 2.1.2.

```
export function heuristic(a: Position, b: Position, type: HeuristicType): number {
  const dx = Math.abs(a.col - b.col);
  const dy = Math.abs(a.row - b.row);
  switch (type) {
    case HeuristicType.MANHATTAN: return dx + dy;
    case HeuristicType.EUCLIDEAN: return Math.sqrt(dx * dx + dy * dy);
    case HeuristicType.CHEBYSHEV: return Math.max(dx, dy);
    case HeuristicType.OCTILE:   return dx + dy + (Math.SQRT2 - 2) * Math.min(dx, dy);
  }
}

export function getMoveCost(grid: Grid, from: Position, to: Position): number {
  const isDiagonal = from.row !== to.row && from.col !== to.col;
  const weight = grid.cells[to.row][to.col].weight;
  return isDiagonal ? weight * Math.SQRT2 : weight;
}
```

Б.3. Бинарни хип са ажурирањем приоритета

Приоритетни ред подржава и промену приоритета већ уписаног чвора, што је потребно зато што информисана претрага може да пронађе јефтинији пут до чвора који се

већ налази у frontiјеру. Положај сваког чвора памти се у помоћној мапи, чиме претрага чвора постаје операција сталне сложености.

```
export class MinHeap {
  private heap: HeapNode[] = [];
  private positionMap: Map<string, number> = new Map();

  push(pos: Position, priority: number): void {
    this.heap.push({ pos, priority });
    const idx = this.heap.length - 1;
    this.positionMap.set(posKey(pos), idx);
    this.bubbleUp(idx);
  }

  updatePriority(pos: Position, newPriority: number): void {
    const idx = this.positionMap.get(posKey(pos));
    if (idx === undefined) return;
    const oldPriority = this.heap[idx].priority;
    this.heap[idx].priority = newPriority;
    if (newPriority < oldPriority) this.bubbleUp(idx);
    else this.bubbleDown(idx);
  }
}
```

Б.4. Мерење цене пута на заједничкој скали

Наредни исечак припада подсистему за мерење и реализује поступак описан у одељку 7.1. Цена се рачуна на путу који је алгоритам вратио, а не преузима се из његове интерне евиденције, чиме поређење различитих поступака постаје могуће.

```
function measurePathCost(grid: Grid, path: [number, number][]): number {
  let total = 0;
  for (let i = 1; i < path.length; i++) {
    const [pr, pc] = path[i - 1];
    const [r, c] = path[i];
    const isDiag = pr !== r && pc !== c;
    total += grid.cells[r][c].weight * (isDiag ? Math.SQRT2 : 1);
  }
  return total;
}
```

ПРИЛОГ В: ИНСТРУМЕНТ ЗА КОРИСНИЧКО ТЕСТИРАЊЕ

Овај прилог садржи потпун инструмент припремљен за испитивање са корисницима, чији је састав описан у одељку 9.4. Испитивање у оквиру овог рада није спроведено, па се инструмент наводи како би га било могуће применити без измена.

В.1. Уводна изјава и упитник о претходном искуству

Испитивање траје око двадесет пет минута. Испитанику се објашњава да се испитује систем, а не он, да не постоји тачан или погрешан начин рада и да у сваком тренутку може да одустане. Уз то се моли да наглас изговара шта покушава и шта очекује. Пре почетка рада испитаник одговара на четири питања. Наводи текућу годину и ниво студија и одговара да ли је слушао предмет на коме се обрађују алгоритми над графовима. Затим наводи да ли је раније користио неки алат за визуелизацију алгоритама и како оцењује сопствено познавање алгоритама за проналажење пута на скали од један до пет.

В.2. Задаци

Задаци се дају један по један, у писаном облику. За сваки се бележи време решавања, да ли је задатак решен без помоћи, уз помоћ или није решен, као и запажања изречена током рада.

Задатак 1. Нацртајте препреку која преграђује мрежу по средини, оставите један пролаз, изаберите алгоритам A^* и покрените визуелизацију.

Задатак 2. Зауставите визуелизацију и пронађите корак у коме је број чворова који чекају обраду највећи. Наведите редни број тог корака.

Задатак 3. Пређите на режим поређења. На мапи која садржи пондерисан терен покрените претрагу у ширину и Дајкстрин алгоритам, па објасните зашто се цене њихових путева разликују.

Задатак 4. Пређите на режим полигона. Изаберите Дајкстрин алгоритам као референтни, конструишите пут од полазног до циљног чвора и предајте решење. Циљ је остварити најмање осамдесет поена.

Задатак 5. Помоћу описа на природном језику затражите мапу на којој се два алгоритма по вашем избору понашају различито, па у приказаној табели проверите да ли је захтев заиста испуњен.

Задатак 6. Сачувајте тренутну мапу под називом који сами изаберете, промените мрежу, а затим поново учитајте сачувану мапу.

В.3. Упитник о употребљивости и завршна питања

Након решавања задатака испитаник попуњава стандардни упитник [29]. Уз сваку тврдњу наводи се оцена на скали од један, што означава потпуно неслагање, до пет, што означава потпуно слагање.

- 1) Мислим да бих овај систем радо користио често.
- 2) Систем ми делује непотребно сложено.
- 3) Систем је био једноставан за коришћење.
- 4) Мислим да би ми била потребна помоћ стручног лица да бих користио овај систем.
- 5) Различити делови система добро су уклопљени у целину.
- 6) Приметио сам превише недоследности у систему.
- 7) Претпостављам да би већина људи брзо научила да користи овај систем.
- 8) Систем ми је деловао гломазно за коришћење.
- 9) Осећао сам се сигурно током коришћења система.
- 10) Морао сам много да учим пре него што сам могао да радим са системом.

Укупна оцена израчунава се на следећи начин. За тврдње са непарним редним бројем од дате оцене одузме се један, а за тврдње са парним редним бројем дата оцена одузме се од пет. Добијене вредности се саберу и помноже са 2,5, чиме се добија резултат у распону од нуле до сто. На крају се испитанику постављају три питања чији се одговори бележе дословно: шта му је у систему било најкорисније, шта му је било најзбуњујуће или најтеже и шта би додао или изменио. За сваког испитаника бележе се ознака, година и ниво студија, податак о слушању предмета о алгоритмима и самопроцена познавања материје. Бележе се и време и исход за сваки од шест задатака, остварени поени у четвртом задатку, укупна оцена упитника и одговори на отворена питања.